

Problema 9.34. Enumerar los quince primeros nodos visitados por un recorrido en anchura del grafo de la figura 9.11, comenzando en el nodo 1 y visitando los vecinos por el orden «primero dividir por 3, luego multiplicar por 2».

Problema 9.35. La Sección 9.5 da una forma de producir el valor 13, empezando desde 1 y utilizando las operaciones de multiplicación por 2 y división por 3. Encontrar otra forma de producir el valor 13 utilizando el mismo punto inicial y las mismas operaciones.

Problema 9.36. Un nodo p de un grafo dirigido $G = \langle N, A \rangle$ se denomina sumidero si para todo nodo $v \in N$, $v \neq p$, existe la arista (v, p) , mientras que no existe la arista (p, v) . Escribir un algoritmo que pueda detectar la presencia de un sumidero en G en un tiempo que esté en $O(n)$, en donde n es el número de nodos que haya en el grafo. El algoritmo debe aceptar el grafo representado por su matriz de adyacencia (del tipo *grafoadya* de la Sección 5.4). Obsérvese que un tiempo de ejecución en $O(n)$ para este problema es algo notable, dado que sólo para escribir los datos se requiere un espacio que está en $\Omega(n^2)$.

Problema 9.37. Un camino de Euler en un grafo no dirigido es un camino en la cual todas las aristas aparecen una sola vez. Diseñar un algoritmo que determine si un grafo dado posee un camino de Euler, y que imprima ese camino si existe. ¿Cuánto tiempo necesita su algoritmo?

Problema 9.38. Repetir el problema 9.37 para un grafo dirigido.

Problema 9.39. Dado un grafo dirigido o no dirigido, se dice que un camino es hamiltoniano cuando pasa exactamente una vez por todos los nodos del grafo, sin volver al nodo inicial. En un grafo dirigido, sin embar-

go, es preciso tener en cuenta la dirección de las aristas en el camino. Demostrar que si un grafo dirigido es completo (esto es, si todo par de nodos está unido al menos en una dirección) entonces tiene un camino hamiltoniano. En este caso, dar un algoritmo para encontrar el camino.

Problema 9.40. Dibujar el árbol de búsqueda explorado por un algoritmo de vuelta atrás que resuelva el mismo caso del problema de la mochila de la Sección 9.6.1, pero suponiendo esta vez que los elementos se cargan en orden decreciente de pesos.

Problema 9.41. Resolver el mismo caso del problema de la mochila de la Sección 9.6.1 por programación dinámica. Habrá que hacer primero el problema 8.5.1.

Problema 9.42. Adaptar la función *mochila* de la Sección 9.6.1 para dar la composición de la carga óptima, además de su valor.

Problema 9.43. Supongamos que el vector $q[1..8]$ representa las posiciones de ocho reinas en un tablero de ajedrez, con una reina en cada fila: si $q[i] = j$, $1 \leq i \leq 8$, $1 \leq j \leq 8$, la reina de la fila i está en la columna j . Escribir una función *solución*(q) que devuelva el valor falso si al menos una de las reinas amenaza a otra, y que devuelva el valor verdadero en caso contrario.

Problema 9.44. Dado que el algoritmo *reinas1* halla una solución y se detiene después de comprobar 1.299.852 situaciones, resolver el problema sin emplear una computadora.

Problema 9.45. Supongamos que el procedimiento *usar*(T) que se invoca desde el procedimiento *perm*(i) de la Sección 9.6 consiste simplemente en imprimir la matriz T en una línea nueva. Mostrar el resultado de llamar a *perm*(1) cuando $n = 4$.

Problema 9.46. Supongamos que el procedimiento $usar(T)$ que se invoca desde el procedimiento $perm(i)$ de la Sección 9.6 requiere un tiempo constante. ¿Cuánto tiempo se necesita, en función de n , para ejecutar la llamada $perm(1)$?

Rehacer el problema suponiendo ahora que $usar(T)$ requiere un tiempo que está en $\Theta(n)$.

Problema 9.47. ¿Para qué valores de n carece de solución el problema de las n reinas? Demostrar la respuesta.

Problema 9.48. ¿Cuántas soluciones tiene el problema de las ocho reinas? ¿Cuántas soluciones *distintas* hay si no distinguimos las soluciones que se pueden transformar unas en otras por rotaciones y reflexiones?

Problema 9.49. Investigar el problema consistente en situar k «reinas» en un tablero tridimensional $n \times n \times n$. Suponga que una «reina» en tres dimensiones amenaza las posiciones de las mismas filas, columnas o diagonales en que se encuentra, de la forma evidente. Claramente, k no puede sobrepasar el valor n^2 . Sin contar el caso trivial $n = 1$, ¿cuál es el valor más pequeño de n para el cual es posible una solución con n^2 «reinas»?

Problema 9.50. Una matriz booleana $M[1..n, 1..n]$ representa un laberinto cuadrado. En general, partiendo de un punto dado, podemos pasar a puntos adyacentes de la misma fila o de la misma columna. Si $M[i, j]$ es *verdadero*, se puede pasar por el punto (i, j) ; si $M[i, j]$ es *falso*, no se puede pasar por el punto (i, j) . La figura 9.20 da un ejemplo.

Dar un algoritmo de vuelta atrás que busque un camino, si existe, desde $(1,1)$ hasta (n, n) . Sin ser completamente formal, por ejemplo, se pueden decir cosas como «para todo punto v que sea un vecino de x hacer...», el algoritmo debe ser claro y preciso.

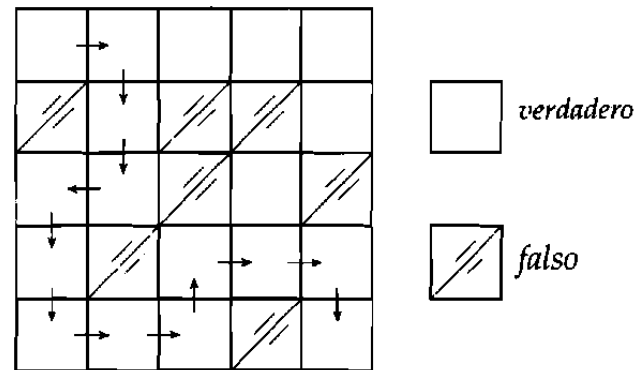


Figura 9.20. Un laberinto

Problema 9.51. El método de vuelta atrás sugerido en el problema 9.50 ilustra el principio, pero resulta muy ineficiente en la práctica. Buscar una forma mucho mejor de resolver el mismo problema.

Problema 9.52. En la Sección 9.7 se calculaban las cotas inferiores de los nodos del árbol de búsqueda suponiendo que toda tarea no asignada sería ejecutada por el agente sin asignación que pudiera hacerlo con menor coste. Esto es lo mismo que tachar las filas y columnas de la matriz de costes que correspondan a los agentes y tareas que ya tengan asignación, y sumar entonces los elementos mínimos de cada una de las columnas restantes. Una forma alternativa de calcular las cotas es suponer que todo agente sin asignación efectuará la tarea que pueda realizar con el mínimo coste. Esto es como tachar las filas y columnas de la matriz de costes correspondientes a los agentes y tareas que ya tengan asignación, y sumar después los elementos mínimos de todas las filas restantes. Mostrar cómo funciona un algoritmo de ramificación y poda para el caso de la figura 9.7 actuando utilizando este segundo método de calcular cotas inferiores.

Problema 9.53. Utilizar un algoritmo de ramificación y poda para resolver los problemas de asignación que tienen asociadas las siguientes matrices:

		1	2	3	4
(a)	<i>a</i>	94	1	54	68
	<i>b</i>	74	10	88	82
	<i>c</i>	62	88	8	76
	<i>d</i>	11	74	81	21

		1	2	3	4	5
(b)	<i>a</i>	11	17	8	16	20
	<i>b</i>	9	7	12	6	15
	<i>c</i>	13	16	15	12	16
	<i>d</i>	21	10	12	11	15
	<i>e</i>	14	10	12	11	15

Problema 9.54. Se dispone de cuatro tipos de objetos, cuyos pesos son respectivamente 7, 4, 3 y 2 unidades, y cuyos valores son 9, 5, 3 y 1. Podemos transportar un máximo de 10 unidades de peso. Los objetos no se pueden fragmentar en trozos. Determinar la carga más valiosa que se pueda

transportar, empleando (a) programación dinámica y (b) ramificación y poda. Para la parte (a), es necesario resolver primero el problema 8.51.

Problema 9.55. Examinando el árbol de la figura 9.18, dijimos que el jugador que va a mover esta situación tiene asegurado un valor de 10 como mínimo. ¿Es esto estrictamente cierto?

Problema 9.56. Supongamos que u corresponde a la situación inicial de las piezas en el juego del ajedrez. ¿Qué se puede decir acerca de $\text{blancas}(u, 12345)$, además del hecho de que tardaríamos demasiado tiempo en calcularlo, incluso empleando una computadora especializada? Justifique su respuesta.

Problema 9.57. El juego de tres en raya tridimensional se juega en un «tablero» $4 \times 4 \times 4$. Tal como en el tres en raya convencional, los jugadores X y O van marcando cuadrados alternativamente. El ganador es el primero que alinea cuatro de sus propias marcas en cualquier dirección, bien sea paralela a un lado del tablero o en una diagonal. Hay 76 maneras de ganar. Diseñar una función de evaluación para situaciones de este juego, y utilizarla para escribir un programa que juegue contra un oponente humano.

9.10 REFERENCIAS Y TEXTOS MÁS AVANZADOS

Existe un buen número de libros acerca de algoritmos para grafos o problemas de combinatoria que suelen plantearse en términos de grafos. Mencionaremos por orden cronológico a Christofides (1975), Lawler (1976), Reingold, Nievergelt y Deo (1977), Gondran y Minoux (1979), Even (1989), Papadimitriou y Steiglitz (1982), Tarjan (1983) y Melhorn (1984b). La noción matemática de grafo se trata extensamente en Berge (1958, 1970).

Berlekamp, Conway y Guy (1982) dan más versiones del juego de Nim que las que interesarán a la mayoría. El juego del problema 9.7 es una variante del juego de Grundy, que también se discute en Berlekamp, Conway y Guy (1982). El libro de Nilsson (1971) es una fuente inagotable de

ideas acerca de grafos y juegos, de la técnica de minimax y de la poda alfa-beta. En Good (1968) aparecen algunos algoritmos para jugar al ajedrez. Una grata descripción de la primera vez que un programa de computadora derrotó a un campeón mundial de *backgammon* se da en Deyong (1977). Para una descripción más técnica de esta hazaña, consúltese Berliner (1980). En 1994, Gari Kasparov, el campeón del mundo de ajedrez, fue derrotado por una microcomputadora Pentium. En el momento de escribir estas líneas, los seres humanos siguen invictos en el juego.

Una solución del problema 9.11 se da en Robson (1973). Para aprender más acerca del precondicionamiento, véase Brassard y Bratley (1988). Se pueden hallar muchos algoritmos basados en el recorrido en profundidad en Tarjan (1972), Hopcroft y Tarjan (1973), y Aho, Hopcroft y Ullman (1974, 1983). El problema 9.24 está resuelto en Hopcroft y Tarjan (1974). Véase también Rosenthal y Goldner (1977) para un algoritmo eficiente que, dado un grafo no dirigido que sea conexo pero no biconexo, busca un conjunto mínimo de aristas que se pueden añadir para hacer que el grafo sea biconexo.

El problema que implica multiplicar por 3 y dividir por 2 nos recuerda el problema de Collatz: véase el problema E16 en Guy (1981). La vuelta atrás se describe en Golomb y Baumert (1965), y se dan técnicas para analizar su eficiencia en Knuth (1975). El problema de las ocho reinas fue inventado por Bezzel en 1848; véase la descripción de Ball (1967). Irving (1984) da un algoritmo de vuelta atrás especialmente eficiente para hallar todas las soluciones del problema de las ocho reinas. El problema se puede resolver en tiempo lineal mediante un algoritmo de divide y vencerás debido a Abramson y Yung (1989), siempre que nos conformemos con una única solución. Volveremos a este problema en el Capítulo 10. El problema tridimensional de n^2 reinas mencionado en el problema 9.49 fue mencionado por McCarty (1978) y resuelto por Allison, Yee y McGaughey (1989): no hay soluciones para $n < 11$ pero existen 264 soluciones para $n = 11$.

La ramificación y poda se explica en Lawler y Wood (1966). El problema de la asignación es muy conocido en investigación operativa: véase Hillier y Lieberman (1967). Para un ejemplo de resolución del problema de la mochila empleando vuelta atrás, véase Hu (1981). El problema del viajante de comercio se resuelve empleando ramificación y poda en Bellmore y Nemhauser (1968).

Algoritmos probabilistas¹

10.1 INTRODUCCIÓN

Imagine que es usted la heroína de un cuento de hadas. Hay un tesoro oculto en un lugar que se describe en un mapa que no consigue descifrar. Se las ha arreglado para reducir la búsqueda a dos posibles escondites, que sin embargo distan bastante entre sí. Si estuviera en uno u otro de estos dos lugares, sabría de inmediato si es o no el correcto. Se necesitan cinco días para ir a cualquiera de los dos lugares, o para viajar desde uno hasta otro. Y el problema se complica más, porque hay un dragón que visita el tesoro todas las noches, y se lleva una parte a un inaccesible cubil en las montañas. Piensa usted que se necesitarán cuatro días de cálculo para resolver el misterio del mapa, y por tanto para saber con certeza dónde está oculto el tesoro, pero si se pone en marcha ya no podrá utilizar una computadora. Un elfo le ofrece mostrarle la forma de descifrar el mapa a cambio del equivalente del tesoro que se lleva el dragón en tres noches. ¿Debe de aceptar la oferta del elfo?

Evidentemente, es preferible dar el valor de tres noches de tesoro al elfo frente a permitir que el dragón se dedique al pillaje cuatro noches más. Sin embargo, si está dispuesta a correr un riesgo calculado, puede hacerlo mejor. Supongamos que es x el valor del tesoro que queda hoy, y que y es el valor del tesoro que se lleva el dragón todas las noches. Supongamos además que es $x > 9y$. Recordando que necesitará cinco días para alcanzar el escondite, puede esperar volver a casa con $x - 9y$ si espera cuatro días para acabar de descifrar el mapa. Si acepta la oferta del elfo, puede partir inmediatamente y volver con $x - 5y$, de lo cual $3y$ servirá para pagar al elfo; entonces le quedará $x - 8y$. Una estrategia mejor consiste en tirar una moneda al aire para decidir cuál de los dos escondites va a visitar primero, viajando hasta el otro si se ha equivocado. Esto le da una oportunidad de entre dos de volver a casa con $x - 5y$, y una oportu-

nidad de entre dos de volver a casa con $x - 10y$. Por tanto, el beneficio esperado es $x - 7,5y$.

Esta fábula se puede traducir al contexto de la algoritmia en la forma siguiente: cuando un algoritmo se enfrenta a una decisión, a veces es preferible seguir un curso de acción aleatorio, en lugar de invertir tiempo en determinar cuál de las alternativas es la mejor. Esta situación se produce cuando el tiempo necesario para determinar la opción óptima es prohibitivo en comparación con el tiempo que se ahorra de análisis en el caso medio al tomar esta decisión óptima.

La característica fundamental de los algoritmos probabilistas es que un mismo algoritmo se puede comportar de forma distinta cuando se aplica dos veces a un mismo caso. Su tiempo de ejecución, e incluso el resultado obtenido, pueden variar considerablemente entre usos consecutivos. Esto se puede explotar de muchas maneras. Por ejemplo, no se permite que un algoritmo determinista pierda el control (bucle infinito, división por cero) porque si hace esto en un caso dado, entonces nunca se podrá resolver ese caso con ese algoritmo. Por contraste, este comportamiento es admisible para un algoritmo probabilista siempre y cuando se produzca con una probabilidad razonablemente pequeña en cualquier caso dado: si el algoritmo se atasca, basta reiniciarlo para ese mismo caso y dispondremos de una nueva oportunidad de éxito. Otra ventaja de este enfoque es que si hay más de una respuesta correcta, se pueden obtener varias diferentes ejecutando el algoritmo probabilista más de una vez; un algoritmo determinista siempre llega a la misma respuesta, aunque por supuesto se puede programar para buscar varias.

Otra consecuencia del hecho de que los algoritmos probabilistas se pueden comportar de forma distinta cuando se ejecutan dos veces con las mismas entradas es que algunas veces les permitiremos que produzcan resultados erróneos. Supuesto que esto suceda con probabilidad suficientemente pequeña en cualquier caso dado, basta invocar el algoritmo varias veces sobre el caso deseado para llegar a una confianza arbitrariamente grande de que se ha obtenido la respuesta correcta. Por contraste, un algoritmo determinista que da respuestas erróneas con algunas entradas es inadmisibles para la mayoría de las aplicaciones, porque *siempre* fallará en esas entradas.

El análisis de algoritmos probabilistas suele ser complicado, y requiere estar familiarizado con resultados de probabilidad y estadística que van más allá de los presentados en la Sección 1.7.4. Por esta razón, en este capítulo se citan varios resultados sin demostración.

10.2 PROBABILISTA NO IMPLICA INCIERTO

Hay muchos algoritmos probabilistas que dan respuestas probabilistas, esto es, respuestas que no son necesariamente exactas. Hay aplicaciones críticas para las cuales no se pueden admitir esas respuestas inciertas. Sin embargo, es frecuente que la probabilidad de error se pueda hacer descender por debajo de la de un

error del equipo físico durante el significativamente mayor tiempo que el necesario para calcular la respuesta de forma determinista. Por tanto, paradójicamente, ¿puede suceder que la respuesta incierta dada por un algoritmo probabilista no sólo se obtenga más deprisa, sino que además podamos confiar más en ella que en la respuesta «garantizada» que se obtiene mediante ningún algoritmo determinista! Además, hay problemas para los cuales no se conoce ningún algoritmo (determinista o probabilista) que pueda dar la respuesta con certeza en una cantidad razonable de tiempo (incluso ignorando posibles errores del equipo físico), y sin embargo hay algoritmos probabilistas que pueden resolver rápidamente el problema si se admite una pequeña probabilidad de error. Tal es el caso, por ejemplo, si se desea determinar si un número de 1.000 dígitos es primo o compuesto.

Hay dos categorías de algoritmos probabilistas que no garantizan la corrección del resultado. Los algoritmos *numéricos* producen un *intervalo de confianza* de la forma «con una probabilidad del 90%, la respuesta correcta es 59 más o menos 3». Cuanto más tiempo se conceda a estos algoritmos numéricos, más preciso es el intervalo. Las encuestas de opinión ofrecen un ejemplo familiar de este tipo de respuesta. Además ilustran que un algoritmo probabilista puede ser mucho más eficiente, aunque menos preciso, que el correspondiente algoritmo determinista (una elección general). Los algoritmos numéricos son útiles si nos basta una aproximación de la respuesta correcta. En contraste, los denominados algoritmos de *Monte Carlo* dan la respuesta exacta con una elevada probabilidad, aunque a veces proporcionan una respuesta incorrecta. En general, no se puede saber si la respuesta proporcionada por un algoritmo de Monte Carlo es correcta, pero se puede reducir arbitrariamente la probabilidad de error dando más tiempo al algoritmo. (Por razones históricas, algunos autores utilizan el término «Monte Carlo» para cualquier algoritmo probabilista, y en particular para los que nosotros denominamos numéricos.)

Sin embargo, no todos los algoritmos probabilistas dan respuestas probabilísticas. Con frecuencia las opciones probabilistas se utilizan solamente como guía para llevar más rápidamente el algoritmo hacia la solución deseada, en cuyo caso la respuesta obtenida siempre es correcta. Además, puede ocurrir que la corrección de toda presunta solución pueda ser verificada eficientemente. Si resulta ser incorrecta, el algoritmo probabilista debería admitir que ha fracasado, en lugar de responder incorrectamente. Los algoritmos probabilistas que nunca dan una respuesta incorrecta se denominan de *Las Vegas*. Una vez más la posibilidad de que el algoritmo pueda admitir un fallo no es dramática: basta seguir aplicando el algoritmo a la entrada deseada hasta que tenga éxito.

Quizá resulte humorístico, pero ilustraremos la diferencia entre los distintos tipos de algoritmos probabilistas por la forma en que responderían a la pregunta ¿cuándo descubrió América Cristóbal Colón? Si lo invocásemos cinco veces, un algoritmo numérico podría responder:

Entre 1490 y 1500
 Entre 1485 y 1495
 Entre 1491 y 1501
 Entre 1480 y 1490
 Entre 1489 y 1499

Si le damos más tiempo al algoritmo, podemos reducir la probabilidad de un intervalo falso (parece ser un 20% en nuestra muestra, puesto que la cuarta respuesta no era correcta) o bien podemos reducir la amplitud de los intervalos (11 en nuestro ejemplo), o las dos cosas. Cuando se le hace la misma pregunta diez veces a un algoritmo de Monte Carlo, podría responder

1492, 1492, 1492, 1491, 1492, 1492, 357 a.C., 1492, 1492, 1492

Una vez más, tenemos una tasa de error del 20% en la respuesta, que se podría hacer más pequeña dando más tiempo al algoritmo. Obsérvese que en algunas ocasiones las respuestas incorrectas están cerca de la respuesta correcta, y a veces son completamente erróneas. Por último, un algoritmo de Las Vegas invocado 10 veces podría responder

1492, 1492, ¡Perdón!, 1492, 1492, 1492, 1492, ¡Perdón!, 1492

El algoritmo nunca da una respuesta incorrecta, pero de vez en cuando no responde.

10.3 TIEMPO ESPERADO FRENTE A TIEMPO PROMEDIO

Haremos una distinción importante entre las palabras «promedio» y «esperado». El tiempo *promedio* de un algoritmo determinista se describió en la Sección 2.4. Se refiere al tiempo promedio requerido por el algoritmo cuando se consideran igualmente probables todos los ejemplares posibles de un tamaño dado. Tal como se vio, el análisis del caso promedio puede inducir a confusión si la realidad es que algunos ejemplares son más probables que otros. Por contraste, el tiempo *esperado* de un algoritmo probabilista se define para cada ejemplar individual: se refiere al tiempo promedio que se necesitaría para resolver ese mismo ejemplar una y otra vez. Ésta es una noción robusta, que no depende de la distribución de probabilidades de los ejemplares que haya que resolver, porque las posibilidades implicadas quedan bajo control directo del algoritmo.

Resulta significativo hablar del tiempo esperado promedio y del tiempo esperado en el caso peor de un algoritmo probabilista. Este último, por ejemplo, se refiere al tiempo esperado que requiere el peor caso posible de un tamaño dado, y *no* al tiempo requerido si lamentablemente se tomasen las peores opciones probabilistas. Los algoritmos de Las Vegas pueden ser más eficientes que los algoritmos deterministas, pero sólo con respecto al tiempo esperado: siempre es posible que la mala fortuna obligue a un algoritmo de Las Vegas a seguir el camino más largo para llegar a una solución. Sin embargo, el comportamiento esperado de los algoritmos de Las Vegas puede ser mucho mejor que el de los algoritmos deterministas.

Por ejemplo, en la Sección 7.5 vimos un algoritmo que puede hallar la mediana de un vector en un tiempo lineal en el caso peor. En la Sección 10.7.2 estudiaremos un algoritmo de Las Vegas para hallar la mediana cuyo tiempo esperado en el caso peor también es lineal con respecto al tamaño del caso. Su rendimiento esperado es considerablemente mejor que el del algoritmo de la Sección 7.5 *en todos y cada uno de los casos*. El único precio que hay que pagar cuando se utiliza este algoritmo de Las Vegas es la improbable posibilidad de que en algunas ocasiones requiera un tiempo excesivo, independientemente del caso en cuestión, y esto no se deberá a otra cosa que a la mala suerte.

Se puede emplear un enfoque parecido para transformar *quicksort* en un algoritmo que ordena n elementos en un tiempo esperado en el caso peor que se encuentra en $O(n \log n)$, mientras que el algoritmo que vimos en la Sección 7.4.2 requiere un tiempo que está en $\Omega(n^2)$ cuando los elementos ya están ordenados. De forma similar, la *dispersión universal* ofrece acceso a una tabla de símbolos en un tiempo esperado constante en el caso peor, mientras que la dispersión clásica resulta desastrosa en el caso peor; véase la Sección 10.7.3. La ventaja principal de hacer aleatorio un algoritmo determinista que ofrezca un buen comportamiento medio a pesar de un caso peor muy malo es hacerlo menos vulnerable a una distribución de probabilidad inesperada de los casos que intente resolver alguna aplicación; véase el final de la Sección 2.4 y la Sección 10.7.

10.4 GENERACIÓN DE NÚMEROS PSEUDOALEATORIOS

A lo largo de este capítulo, suponemos que está disponible un generador de números aleatorios que se puede invocar con coste unitario. Sean a y b dos números reales tales que $a < b$. Una llamada a *uniforme*(a, b) devuelve un número real x seleccionado aleatoriamente en el intervalo $a \leq x < b$. La distribución de x es uniforme en el intervalo, y las sucesivas llamadas al generador devuelve valores independientes de x . Para generar enteros aleatorios, extendemos la notación a *uniforme*($i..j$), en donde i y j son enteros, $i \leq j$, y la llamada devuelve un entero k seleccionado aleatoriamente, uniformemente e independientemente dentro del intervalo $i \leq k \leq j$. Un caso especial útil es la función *tiramonedá*, que devuelven *cara* o *cruz*, cada una de ellas con una probabilidad del 50%. Obsérvese que *uniforme*(a, b), *uniforme*($i..j$) y *tiramonedá* son fáciles de obtener aun cuando sólo esté disponible *uniforme*(0, 1):

función *uniforme*(a, b)

devolver $a + (b - a) \times \text{uniforme}(0, 1)$

función *uniforme*($i..j$)

devolver $\lfloor \text{uniforme}(i, j + 1) \rfloor$

función *tiramonedá*

si *uniforme*(0..1) = 0 **entonces devolver** *cara*
sino devolver *cruz*

Los generadores verdaderamente aleatorios no suelen estar disponibles en la práctica. Además, no es realista esperar que $\text{uniforme}(0, 1)$ tome un valor *real* arbitrario entre 0 y 1. En la mayoría de las ocasiones, se emplean generadores *pseudoaleatorios*: se trata de procedimientos deterministas capaces de generar largas sucesiones de valores que parecen tener las propiedades de una sucesión aleatoria. Para dar comienzo a una sucesión, proporcionamos un valor inicial denominado *semilla*. Una misma semilla da lugar siempre a la misma sucesión. Para obtener sucesiones diferentes, podemos seleccionar, por ejemplo, una semilla que dependa de la fecha o de la hora. Casi todos los lenguajes de programación contienen uno de estos generadores, aunque hay que utilizar con precaución algunas implementaciones. Empleando un buen generador pseudoaleatorio, podemos esperar que sean válidos los resultados teóricos obtenidos en este capítulo concernientes a la eficiencia y fiabilidad de diferentes algoritmos probabilistas. Sin embargo, la poco práctica hipótesis de que está disponible un generador verdaderamente aleatorio resulta crucial cuando se efectúa el análisis.

La teoría de los generadores pseudoaleatorios va más allá de los límites de este libro, pero el principio general es sencillo. La mayoría de los generadores está basada en una pareja de funciones $f: X \rightarrow X$ y $g: X \rightarrow Y$, donde X es un conjunto suficientemente grande e Y es el dominio de los valores pseudoaleatorios que haya que generar. Se impone que tanto X como Y sean finitos, lo cual significa que sólo podemos tener la esperanza de aproximar el efecto de $\text{uniforme}(0, 1)$ empleando un Y adecuadamente grande. Por otra parte, $Y = \{0, 1\}$ resulta adecuado para muchas aplicaciones: corresponde a tirar una moneda imparcial. Para generar nuestra secuencia pseudoaleatoria, sea $s \in X$ una semilla. Esta semilla define una secuencia:

$$\begin{cases} x_0 = s \\ x_i = f(x_{i-1}) \text{ para todo } i > 0 \end{cases}$$

Finalmente, la sucesión pseudoaleatoria $y_0, y_1, y_2, \dots$ se define de la forma $y_i = g(x_i)$ para todo $i \geq 0$. Esta sucesión es esencialmente periódica, con un periodo que no puede sobrepasar el número de elementos que hay en X . Sin embargo, si X es suficientemente grande y si f y g (y a veces s) se seleccionan adecuadamente, entonces el periodo puede hacerse muy grande, y la sucesión puede ser, para casi todos los efectos prácticos, indistinguible de una sucesión verdaderamente aleatoria de elementos de Y .

Daremos un ejemplo sencillo para ilustrar este principio. Sean p y q dos números primos muy grandes distintos, ambos congruentes con 3 módulo 4, y sea n su producto. Supongamos que p y q son tan grandes que no resulta factible factorizar n . (En el momento de escribir esto, se consideran suficientes 100 cifras para cada número; véase la Sección 10.7.4.) Sea X el conjunto de enteros desde 0 hasta $n-1$, y sea $Y = \{0, 1\}$. Definimos $f(x) = x^2 \bmod n$ y $g(x) = x \bmod 2$. Siempre y cuando la semilla se seleccione uniformemente al azar entre los elementos de X que son relativamente primos con respecto a n , se ha demostrado que casi siempre es imposible distinguir la sucesión generada de esta manera de una sucesión binaria verdaderamente aleatoria. Para la mayoría de los propósitos algorítmicos, pueden resultar preferibles unos generadores pseudoaleatorios más rápidos pero menos seguros. Por ejemplo,

gozan de amplia aceptación los generadores pseudoaleatorios de congruencia lineal de la forma $f(x) = ax + b \bmod n$, para valores adecuados de a , b y n .

10.5 ALGORITMOS PROBABILISTAS NUMÉRICOS

La aleatoriedad se utilizó por primera vez en algoritmia para resolver aproximadamente problemas numéricos. Ya se utilizaba en el mundo secreto de la investigación atómica durante la Segunda Guerra Mundial, momento en que se dio a esta técnica el nombre de «Monte Carlo» (término que algunos autores utilizan todavía para denotar los algoritmos probabilistas numéricos, pero que emplearemos con un significado diferente en este libro). Un ejemplo familiar de un algoritmo numérico probabilista es la simulación. Esta técnica se puede emplear, por ejemplo, para estimar la longitud media de la cola en un sistema tan complejo que no sea factible obtener soluciones en forma cerrada, o incluso respuestas numéricas por métodos deterministas. Para ciertos problemas de la vida real, el cálculo de una solución exacta no es posible ni siquiera en principio, quizá debido a incertidumbres en los datos experimentales que haya que utilizar, o quizá porque una computadora digital sólo puede manejar valores binarios o decimales mientras que la respuesta que hay que calcular es irracional. Para otros problemas, existe una respuesta precisa pero se tardaría demasiado tiempo en calcularla exactamente.

La respuesta obtenida mediante un algoritmo probabilista numérico siempre es aproximada, pero su precisión esperada mejora a medida que crece el tiempo disponible para el algoritmo. El error suele ser inversamente proporcional a la raíz cuadrada de la cantidad de trabajo que hayamos efectuado, así que se necesita 100 veces más trabajo para obtener un dígito de precisión adicional.

10.5.1 La aguja de Buffon

En el siglo XVIII, Georges Louis Leclerc, conde de Buffon, demostró que si se arroja aleatoriamente una aguja (en una posición aleatoria, con un ángulo aleatorio, con una distribución uniforme) sobre un suelo hecho con planchas de anchura constante, si la aguja tiene una longitud que sea exactamente la mitad de la anchura de las planchas, y si la anchura de la rendija existente entre las planchas es cero, entonces la probabilidad de que una aguja caiga encima de una rendija es $1/\pi$. Este teorema puede servir para predecir aproximadamente cuántas agujas caerán encima de una rendija si dejamos caer n de ellas: la esperanza es n/π . La figura 10.1 ilustra la caída de 22 agujas, de las cuales 8 han caído encima de una rendija; nosotros habríamos esperado 7 análisis en el caso medio, puesto que es $\pi \approx 22/7$. Suponemos que las agujas se dejan caer al azar e independientemente unas de otras: para evitar interferencias, no deben superponerse, ni siquiera deben tocarse. Quizá sea mejor tirar y recoger la misma aguja varias veces.

Sin embargo, existe una interpretación del teorema de Buffon que resulta algorímicamente más interesante. En lugar de utilizarlo para predecir el número de agujas que caerán encima de una rendija (reconocemos que esto no tiene demasiado interés), ¿por qué no utilizarlo al revés, para estimar el valor de π ? Suponga

que no dispone de una buena aproximación de π , pero conoce el teorema de Buffon. Imagine que deja caer un gran número de agujas, n , en el suelo, y que cuenta el número de agujas k que caen encima de una rendija. Ahora puede utilizar n/k como estimación de π . (Sería tener mala suerte si $k = 0$, pero esto resulta muy poco probable cuando n es grande.) Cuanto más precisa quiera que sea su estimación, más agujas tendrá que dejar caer. En principio, puede aproximarse cuanto quiera al valor de π sin más que dejar caer un número suficiente de agujas, o preferiblemente dejar caer una misma aguja un número suficiente de veces para evitar interferencias. Llevaremos a cabo un análisis más preciso más adelante en esta sección. En la práctica, esto no es un «algoritmo» útil, porque se pueden obtener aproximaciones de π mucho mejores empleando métodos deterministas. Sin embargo, esta aproximación a la «determinación experimental de π » fue muy utilizada en el siglo XIX, haciendo de éste uno de los primeros algoritmos probabilistas que se utilizaron.

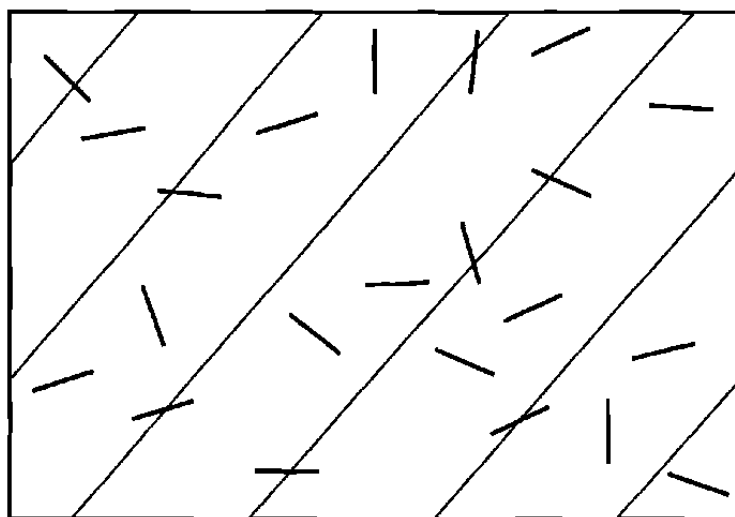


Figura 10.1 La aguja de Buffon

Hemos visto que el teorema de Buffon puede servir para predecir el número de agujas que caen encima de una rendija, y para estimar el valor de π . Una tercera aplicación posible para este teorema es estimar la anchura de las planchas (en términos de la longitud de la aguja). Recordemos que las dos aplicaciones anteriores daban por supuesto que las agujas son *precisamente* de longitud igual a la mitad de la anchura de las planchas. Si no es cierta esta suposición, entonces ya no es cierto que n/k se aproxime arbitrariamente a π a medida que vayamos tirando más y más agujas. Sean w y λ la anchura de las planchas y la longitud de las agujas, respectivamente. Buffon demostró un teorema más general: supuesto que sea $w \geq \lambda$, la probabilidad de que una aguja tirada aleatoriamente caiga encima de una rendija es $p = 2\lambda/w\pi$. En particular, $p = 1/\pi$ cuando $\lambda = w/2$, tal como hemos supuesto hasta el momento. Supongamos ahora que conocemos tanto π como la longitud de las agujas precisamente, pero que no tenemos a mano una regla para medir la

longitud de las planchas. Primero nos aseguramos de que las agujas sean más cortas que la anchura de las planchas. En este (¡improbable!) escenario, podemos estimar la anchura de las planchas con una precisión arbitraria dejando caer un número de agujas n suficientemente grande, y contando el número k de agujas que caen encima de una rendija. Nuestra estimación es, simplemente, $w \approx 2\lambda n/k\pi$. Como siempre, cuantas más agujas tiremos, más precisa será nuestra estimación.

Con una probabilidad elevada, los algoritmos para estimar el valor de π y la anchura de las planchas proporcionan un valor que converge a la respuesta correcta a medida que el número de agujas tiradas tiende a infinito (siempre y cuando las agujas tengan una longitud igual a la mitad de las planchas en el primer caso, y conociendo con precisión π y la longitud de las agujas en el segundo). La pregunta natural que se hace uno es: ¿con qué velocidad convergen estos algoritmos? Lamentablemente, son muy lentos: se necesita dejar caer la aguja 100 veces más que hasta el momento para obtener un dígito decimal adicional de precisión.

El análisis de la convergencia de los algoritmos probabilistas numéricos requiere más conocimientos de probabilidad y de estadística que los que normalmente supondríamos por parte de nuestros lectores. Sin embargo, vamos a seguir con un esbozo del enfoque básico para aquellos que tengan los conocimientos necesarios. Nos concentraremos en el análisis del algoritmo para estimar π . Consideremos un ε positivo y arbitrariamente pequeño. Asociamos una variable aleatoria X_i a cada aguja: $X_i = 1$ si la i -ésima aguja cae encima de una rendija, y $X_i = 0$ en caso contrario. Por el teorema de Buffon, $\Pr[X_i = 1] = 1/\pi$ para cada i . La esperanza y varianza de X_i son $E(X_i) = 1/\pi$ y $\text{Var}(X_i) = \frac{1}{\pi}(1 - \frac{1}{\pi})$, respectivamente. Ahora, supongamos que X representa la variable aleatoria correspondiente a nuestra estimación de $1/\pi$ después de dejar caer n agujas: $X = \sum_{i=1}^n X_i / n$. Para cualquier entero k entre 0 y n ,

$$\Pr[X = k/n] = \binom{n}{k} \left(\frac{1}{\pi}\right)^k \left(1 - \frac{1}{\pi}\right)^{n-k}.$$

Por ejemplo, cuando $n = 22$, $\Pr[X = \frac{7}{22}] \approx 18\%$ y $\Pr[X = \frac{6}{22} \leq X \leq \frac{8}{22}]$ es un poquito superior al 50%. Esta variable aleatoria tiene una esperanza $E(X) = 1/\pi$ y una varianza $\text{Var}(X) = \frac{1}{\pi}(1 - \frac{1}{\pi})/n$. Por el Teorema Central del Límite, la distribución de X es casi normal siempre y cuando n sea suficientemente grande; véase la Sección 1.7.4. Las tablas de la distribución normal nos dicen que es

$$\Pr[|X - E(X)| < 1,645 \sqrt{\text{Var}(X)}] \approx 90\%.$$

Utilizando nuestros valores para $E(X)$ y para $\text{Var}(X)$, inferimos que $\Pr[|X - \frac{1}{\pi}| < \varepsilon] \geq 90\%$, cuando

$$\varepsilon > 1,645 \sqrt{\frac{1}{\pi} \left(1 - \frac{1}{\pi}\right) / n}$$

y por tanto cuando $n > 0,5881 / \varepsilon^2$. Por tanto, basta con dejar caer al menos $0,5881 / \varepsilon^2$ agujas para obtener un error absoluto en nuestra estimación de $1/\pi$ que no sobre-

pase ε nueve veces de cada diez. Esta dependencia de $1/\varepsilon^2$ explica la razón por la cual un dígito adicional de precisión —que significa que ε tiene que ser 10 veces menor— requiere 100 veces más agujas. Si no estamos satisfechos con un intervalo de confianza que sólo resulta fiable nueve veces de cada diez, se puede utilizar otra entrada de la tabla de distribución normal. Por ejemplo, el hecho de que

$$\Pr [|X - E(X)| < 2,576 \sqrt{\text{Var}(x)}] \approx 99\%$$

nos dice que $\Pr [|X - \frac{1}{\pi}| < \varepsilon] \approx 99$ supuesto que, $\varepsilon > 2,576 \sqrt{\frac{1}{\pi} (1 - \frac{1}{\pi}) / n}$ lo cual se satisface cuando $n > 1,440 / \varepsilon^2$. Por tanto, una reducción de diez veces en la probabilidad de error resulta poco generosa en comparación con un aumento de una cifra de precisión. En general, una vez que hayamos decidido cuántas agujas vamos a dejar caer, existe una compensación entre el número de dígitos que obtenemos y la probabilidad de que esos dígitos sean correctos.

Por supuesto, nos interesa hallar un intervalo de confianza para nuestra estimación de π , y no de $1/\pi$. Un cálculo sencillo muestra que si $|X - \frac{1}{\pi}| < \varepsilon$ entonces $|\frac{1}{X} - \pi| < \frac{\varepsilon\pi^2}{1 - \varepsilon\pi}$. Y además $9,8 \varepsilon < \frac{\varepsilon\pi^2}{1 - \varepsilon\pi} < 10\varepsilon$ cuando $\varepsilon < 0,00415$, lo cual significa intuitivamente que un mismo número de agujas proporcionará una cifra decimal menos respecto a π que respecto a $1/\pi$, aun cuando el error relativo sea esencialmente el mismo puesto que π es aproximadamente 10 veces mayor que $1/\pi$. Resumiendo todo esto, suponiendo que necesitemos un error absoluto en nuestra estimación de π que no sobrepase $\varepsilon < 0,0415$ al menos el 99% de las veces, basta con dejar caer $144/\varepsilon^2$ agujas. Bastan unas pocas agujas menos en el límite de ε pequeño. ¡Esto es casi un millón y medio de agujas aunque nos conformemos con que nuestra estimación diste menos de 0,01 respecto al valor exacto de π en el 99% de las ocasiones! Nunca dijimos que esta aproximación fuera práctica, ¿verdad?

Para resumir, el usuario del algoritmo debería proporcionar dos parámetros: la fiabilidad deseada ρ , y la precisión ε . A partir de estos parámetros, con ayuda de una tabla de distribución normal, el algoritmo determina el número n de agujas que es necesario dejar caer para que la estimación resultante esté entre $\pi - \varepsilon$ y $\pi + \varepsilon$ con una probabilidad ρ como mínimo. Por ejemplo, el algoritmo seleccionaría un n próximo a un millón y medio si el usuario indicase $\rho = 99\%$ y $\varepsilon = 0,01$. Por último, el algoritmo deja caer n agujas al suelo, cuenta el número k de agujas que caen encima de una rendija, y proclama: «El valor de π está probablemente entre $\frac{n}{k} - \varepsilon$ y $\frac{n}{k} + \varepsilon$ ». Esta respuesta será correcta una fracción ρ de las veces si se repite el experimento varias veces con los mismos valores de ρ y de ε .

Queda una sutileza si se desea implementar un algoritmo similar para estimar un valor que no se conozca de antemano: ¡necesitamos el valor de π para nuestro análisis de convergencia! ¿Cómo podemos determinar el número requerido de agujas, n , como función únicamente de ρ y de ε ? Una solución es ser muy conservador en nuestra estimación de la varianza de las variables aleatorias implicadas. La varianza de X_i es $p(1-p)$ cuando $X_i = 1$ con probabilidad p , y $X_i = 0$ en caso contrario, lo cual es casi un cuarto (el caso peor es aquél en el que $p = 1/2$). Si determinamos n como si la varianza de X_i fuese un cuarto, entonces dejaremos caer más agujas que las estrictamente necesarias, pero la precisión deseada se obtendrá con una fiabilidad mayor que la re-

querida. Entonces el algoritmo puede estimar la fiabilidad real obtenida empleando una tabla de la distribución t de Student. Los detalles se pueden encontrar en cualquier texto estandar de estadística. Una solución alternativa es utilizar una pequeña muestra de tamaño arbitrario para obtener una primera estimación del valor requerido, utilizar esto para hacer una estimación mejor del tamaño de muestra necesario, y así sucesivamente. Una vez más, los detalles están en cualquier texto estándar.

10.5.2 Integración numérica

Esto nos lleva al más conocido de los algoritmos probabilistas numéricos: la integración de Monte Carlo, cuyo nombre es poco afortunado porque *no* es un ejemplo de algoritmo de Monte Carlo según la terminología empleada en este libro. Recordemos que si $f: \mathbb{R} \rightarrow \mathbb{R}^{\geq 0}$ es una función continua y si $a \leq b$, entonces el área de la superficie limitada por la curva $y = f(x)$, el eje x y las líneas verticales $x = a$ y $x = b$ es

$$I = \int_a^b f(x) dx$$

Consideremos ahora un rectángulo de anchura $b - a$ y de altura $I/(b - a)$ situado tal como indica la figura 10.2. El área del rectángulo también es I . Dado que el rectángulo y la superficie bajo la curva tienen la misma área, tienen que tener la misma altura media. Concluimos que la altura media de la curva entre a y b es $I/(b - a)$. Si consideramos que los puntos que están bajo el eje X tienen altura negativa, entonces esta interpretación es válida más generalmente para funciones continuas $f: \mathbb{R} \rightarrow \mathbb{R}$.

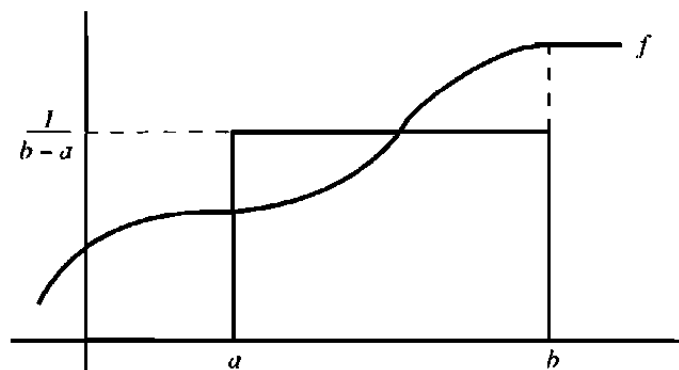


Figura 10.2 Integración numérica

Después de leer la sección anterior, un algoritmo probabilista para estimar la integral debería ocurrirnos inmediatamente: estimar la altura media de la curva por muestreo aleatorio y multiplicar el resultado por $b-a$.

```

función intMC( $f, n, a, b$ )
   $\text{suma} \leftarrow 0$ 
  para  $i \leftarrow 1$  hasta  $n$  hacer
     $x \leftarrow \text{uniforme}(a, b)$ 
     $\text{suma} \leftarrow \text{suma} + f(x)$ 
  devolver  $(b-a) \times (\text{suma}/n)$ 

```

Un análisis similar al de la aguja de Buffon muestra que la varianza de la estimación calculada mediante este algoritmo es inversamente proporcional al número de puntos de la muestra, y que la distribución de la estimación es aproximadamente normal cuando n es muy grande. Por tanto, el error esperado en la estimación es de nuevo inversamente proporcional a $\sqrt{n}$, así que se necesita 100 veces más trabajo para obtener un dígito adicional de precisión.

El algoritmo *intMC* es apenas más práctico que el método de Buffon para estimar π ; véase el problema 10.3. En general, se puede obtener una estimación mejor de la integral mediante métodos deterministas. Quizá el más sencillo es de espíritu similar a *intMC*, salvo que estima la altura media por muestreo determinista a intervalos regulares:

```
función intDET(f, n, a, b)
    suma  $\leftarrow$  0
    delta  $\leftarrow$  (b - a) / n
    x  $\leftarrow$  a + delta / 2
    para i  $\leftarrow$  1 hasta n hacer
        suma  $\leftarrow$  suma + f(x)
        x  $\leftarrow$  x + delta
    devolver suma  $\times$  delta
```

En general, este algoritmo determinista necesita muchas menos iteraciones que la integración de Monte Carlo para obtener un grado de precisión comparable. Esto es típico de la mayoría de las funciones que podamos desear integrar. Sin embargo, a todo algoritmo determinista de integración, incluso al más sofisticado, le corresponden funciones continuas diseñadas expresamente para engañar al algoritmo. Considérese por ejemplo la función $f(x) = \sin^2((100!) \pi x)$. Toda llamada a *intDET*(*f*, *n*, 0, 1) con $1 \leq n \leq 100$ devuelve el valor 0, aun cuando el valor real de esta integral es $1/2$. Ninguna función puede hacerle esta jugada al algoritmo de Monte Carlo, aunque existe una probabilidad extremadamente pequeña de que el algoritmo pudiera cometer un error similar aun cuando *f* sea una función completamente normal.

Una razón mejor para utilizar la integración de Monte Carlo surge en la práctica cuando se tiene que evaluar una integral múltiple. Si se generaliza un algoritmo de integración determinista que utilice algún método sistemático para muestrear la función, adecuándolo a varias dimensiones, entonces el número de puntos de muestra necesarios para lograr una precisión dada crece de manera exponencial con la dimensión de la integral que haya que evaluar. Si se necesitan 100 puntos para evaluar una integral simple, entonces es probable que sea necesario utilizar los 10.000 puntos de una retícula 100×100 para alcanzar la misma precisión cuando se evalúe una integral doble; se necesitará un millón de puntos para una integral triple, y así sucesivamente. Por otra parte, empleando la integración de Monte Carlo, la dimensión de la integral suele tener poco efecto sobre la precisión obtenida, aunque es probable que la cantidad de trabajo para cada iteración aumente ligeramente con la dimensión. En la práctica, la integración de Monte Car-

lo se utiliza para evaluar integrales de cuatro o más dimensiones, porque no hay ninguna otra técnica sencilla que sea competitiva. Sin embargo, hay algoritmos mejores, que son más complicados. Por ejemplo, se puede mejorar la precisión de la respuesta empleando técnicas híbridas que son parcialmente sistemáticas y parcialmente probabilistas. Si la dimensión es fija, puede ser preferible seleccionar sistemáticamente los puntos de tal modo que estén igualmente espaciados y sea razonable su número; esta técnica se conoce con el nombre de *cuasi* Monte Carlo.

10.5.3 Conteo probabilista

¿Hasta dónde se puede contar con un registro de n bits? Empleando la notación binaria ordinaria, se puede contar desde 0 hasta $2^n - 1$. ¿Se puede ir más allá? Claramente, el registro no puede suponer más de 2^n valores distintos, puesto que ése es el número de formas distintas de n cifras binarias. Sin embargo, podríamos ir más allá de 2^n si estuviéramos dispuestos a descartar valores intermedios. Por ejemplo, se podría contar el doble si estuviéramos dispuestos a contar solamente los valores pares. Más exactamente, sea c nuestro registro de n bits. Algunas veces utilizaremos también c para denotar el entero que representa el registro en binario. Deseamos implementar dos procedimientos *iniciar*(c) y *pulsar*(c), y una función *contar*(c) tal que una llamada a *contar*(c) devuelva el número de llamadas a *pulsar*(c) habidas desde la última llamada a *iniciar*(c). En otras palabras, *iniciar* pone el contador a cero, *pulsar* le suma 1 y *contar* pide su valor actual. Los algoritmos deben ser capaces de mantener un número arbitrariamente grande de contadores $c_1, c_2, \dots$, y no se admiten los efectos secundarios: no se puede pasar información entre llamadas, salvo a través del registro que se transmite como parámetro explícito.

Sugeríamos más arriba que se podían saltar algunos valores para llegar más lejos. Sin embargo, esto carece de sentido si insistimos en utilizar una estrategia *determinista* de recuento. Dado que no se admiten efectos secundarios, no hay forma de que *pulsar* pueda sumar 1 al contador en llamadas alternativas: el comportamiento de *pulsar*(c) debe de estar completamente determinado a partir del valor actual de c . Si existe un valor de c tal que *pulsar*(c) deja intacto a c , entonces el contador se detendrá en ese punto hasta que se llame a *iniciar*(c). (Esto es razonable cuando el contador ha alcanzado su límite superior.) Dado que c sólo puede admitir 2^n valores diferentes, el contador se ve obligado a tomar un valor que ya haya tenido anteriormente al cabo de un máximo de 2^n llamadas a *pulsar*(c). Por tanto, es imposible contar de manera determinista más de 2^n sucesos en un registro de n bits.

Si relajamos el requisito consistente en que *contar* debe de devolver el número exacto de llamadas a *pulsar* desde el último *iniciar*, existe una estrategia probabilista evidente para contar dos veces más. El registro es un contador binario normal, que se pone a cero mediante una llamada a *iniciar*. Cada vez que se llama a *pulsar*, lanzamos al aire una moneda imparcial. Si sale cara, sumamos 1 al registro. Si sale cruz, sin embargo, no hacemos nada. Cuando se llama a *contar*, devuelve el doble del valor almacenado en el registro. Después de una llamada a *iniciar*, es fácil demostrar que el valor esperado que proporciona *contar* después de t llamadas

a pulsar es precisamente t , aun cuando t sea impar. Se puede demostrar que la varianza es razonablemente pequeña, así que la estimación proporcionada por *contar* es bastante precisa en la mayoría de las ocasiones; véase el problema 10.5.

No esperamos impresionarle con el párrafo anterior. Contar dos veces más significa simplemente llegar hasta $2^{n+1} - 2$, lo cual se podría haber logrado de forma determinista si hubiera estado disponible un solo *bit* más dentro del registro. Ahora mostraremos que las estrategias de conteo probabilista pueden llegar *exponencialmente* más allá: desde 0 hasta $2^{2^c-1} - 1$. Por tanto, bastan 8 *bits* para contar ¡más de 5×10^{76} sucesos! La idea consiste en mantener en el registro una estimación del logaritmo del número real de llamadas a pulsar. Más exactamente, *contar*(c) devuelve $2^c - 1$. Le restamos 1 para que se pueda representar un conteo de 0; además *contar*(0) = 0, tal como debe de ser, porque *iniciar*(c) pone c a 0. Queda por ver la forma en que se implementa *pulsar*.

Supongamos que $2^c - 1$ es una buena estimación del número de llamadas a pulsar desde la última iniciación. Después de un adicional a pulsar, una buena estimación del número total de pulsos sería 2^c , pero esto no se puede representar así con nuestra estrategia. Para soslayar el problema, sumaremos 1 a c con alguna probabilidad p que ya se determinará, y dejaremos c intacto en caso contrario. De esta manera, nuestra estimación del número de llamadas a pulsar pasa a ser $2^{c+1} - 1$ con probabilidad p , mientras que tiene una probabilidad $1 - p$ la posibilidad de que el contador persista con su valor $2^c - 1$. El valor esperado que devuelve *contar*(c) después de esta llamada a pulsar es por tanto

$$(2^{c+1} - 1)p + (2^c - 1)(1 - p) = 2^c + 2^c p - 1$$

y se ve que basta hacer $p = 2^{-c}$ para obtener 2^c , que es el valor esperado que se deseaba para *contar*(c). (Este razonamiento no es riguroso; hacer el problema 10.6 para una demostración de que el valor esperado que devuelve *contar* es el correcto.) Para resumir, obtenemos los algoritmos siguientes:

procedimiento *iniciar*(c)

$c \leftarrow 0$

procedimiento *pulsar*(c)

para $1 \leftarrow 1$ **hasta** n **hacer**

si *tiramoneda* = *cara* **entonces** **terminar**

$c \leftarrow c + 1$

{La probabilidad de desbordamiento es demasiado pequeña para que valga la pena comprobarlo }

función *contar*(c)

devolver $2^c - 1$

Al guardar el logaritmo del conteo actual en c , se obtiene una ventaja adicional. Vimos en las secciones anteriores que los algoritmos probabilistas numéricos mantenían bajo control el error absoluto. Cuando se implementa un contador cuyo

rango alcance es tan grande como 5×10^6 , es más probable que se necesite mantener bajo control el error *relativo*. Afortunadamente, un pequeño error absoluto en el logaritmo del número de llamadas a pulsar se traduce en un pequeños error relativo del contador real. En particular, se garantiza que *contar* dará la respuesta exacta cuando el número de pulsos habidos desde la inicialización es 0 o 1. Se puede probar por inducción matemática que la varianza de *contar* al cabo de m llamadas a pulsar es $m(m-1)/2$. Esto no está demasiado bien, porque significa que la desviación estándar es alrededor del 70% del valor real de conteo: este proceso distinguiría un millón de mil millones, pero no sería fiable para distinguir un millón de dos millones.

Aunque hay aplicaciones para las cuales una carencia de precisión semejante resulta aceptable, ninguna aplicación real puede tener necesidad de contar hasta 5×10^6 sucesos. Afortunadamente, la varianza del conteo estimado se puede mejorar a costa de no llegar tan lejos al contar. Para esto, basta llevar la cuenta en el registro del logaritmo en una base que sea menor que dos. Por ejemplo, podríamos utilizar

$$\text{contar}(c) = \left[(1 + \varepsilon)^c - 1 \right] / \varepsilon \quad (10.1)$$

para ε pequeño. La división por ε es para mantener la propiedad deseable de que *contar* siempre da la respuesta exacta al cabo de 0 o 1 llamadas a pulsar. Con $\varepsilon = 1/30$, esto nos permite contar hasta más de 125.000 sucesos en un registro de 8 *bits*, con un error relativo menor que el 24% durante el 95% del tiempo. Por supuesto, la probabilidad de que *pulsar* vaya a incrementar c cuando sea invocado ya no debe de ser 2^{-1} : hay que darle un valor adecuado; véase el problema 10.8.

Para un tipo de conteo probabilista completamente diferente que ahorra tiempo en lugar de espacio, asegúrese de hacer el problema 10.12. En ciertas condiciones, este algoritmo puede contar aproximadamente el número de elementos que hay en un conjunto en un tiempo que esta en el orden de la raíz cuadrada de ese número.

10.6 ALGORITMOS DE MONTE CARLO

Existen problemas para los cuales no se conoce ningún algoritmo eficiente (probabilista o determinista) que sea capaz de obtener una solución correcta en todas las ocasiones. Los algoritmos de *Monte Carlo* cometen ocasionalmente un error, pero encuentran la solución correcta con una probabilidad alta sea cual sea el caso considerado. Esto es una afirmación más fuerte que decir que funciona correctamente en la mayoría de los casos, fallando tan sólo de vez en cuando en algunos casos especiales: no debe haber ningún caso en el cual la probabilidad de error sea elevada. Sin embargo, no suele darse ningún aviso cuando el algoritmo comete un error.

Sea p un número real tal que $0 < p < 1$. Decimos que un algoritmo de Monte Carlo es p -correcto si devuelve una respuesta correcta con probabilidad no menor que p , sea cual sea el caso considerado. En algunos casos, permitiremos que p dependa del tamaño del caso, pero nunca del caso en sí. La característica más importante de los algoritmos de Monte Carlo es que suele ser posible reducir arbitrariamente la probabilidad de error a costa de un ligero aumento del tiempo de cálculo. Llamamos a esto *amplificación de la ventaja estocástica*. Estudiaremos este fenómeno general en la Sección 10.6.4, pero primero consideramos dos ejemplos. El primero tiene escaso interés práctico, pero tiene la ventaja de su sencillez. Lo utilizamos para presentar las nociones clave. El segundo, por otra parte, tiene un indudable interés práctico.

10.6.1 Verificación de la multiplicación de matrices

Consideremos tres matrices $n \times n$, A , B y C . Se sospecha que $C = AB$. ¿Cómo se puede verificar si esto es o no verdad? La aproximación evidente consiste en multiplicar A por B , y comparar el resultado con C . Empleando un sencillo algoritmo de multiplicación matricial, esto requiere un tiempo que está en $\Theta(n^3)$. El algoritmo de Strassen es más rápido, y existen algoritmos de multiplicación aún más rápidos para valores muy grandes de n , pero el algoritmo asintóticamente más rápido conocido hasta el momento sigue requiriendo un tiempo que está en $\Omega(n^{2.37})$; véase la Sección 7.6. ¿Podremos hacerlo mejor si estamos dispuestos a admitir una pequeña probabilidad de error? Sorprendentemente, la respuesta es que para cualquier $\varepsilon > 0$ prefijado, basta un tiempo que está en $O(n^2)$ para resolver este problema con una probabilidad de error que sea como mucho ε . La constante oculta en la notación O , sin embargo, depende de ε .

Supongamos que $C \neq AB$. Sea $D = AB - C$. Por hipótesis, D no es idénticamente nulo. Sea i un entero tal que la i -ésima fila de D contiene al menos un elemento no nula. Consideremos cualquier subconjunto $S \subseteq \{1, 2, \dots, n\}$. Supongamos que $\sum_S(D)$ representa el vector de longitud n que se obtiene sumando componente a componente las filas de D indexadas por los elementos de S ; una suma vacía es igual a cero. Por ejemplo, $\sum_{\{1, 3, 4\}}(D)$ se obtiene sumando las filas primera, tercera y cuarta de D . Sea S' el mismo subconjunto que S , salvo que $i \in S'$ si y sólo si $i \notin S$. Dado que la i -ésima fila de D no es idénticamente 0, $\sum_S(D)$ y $\sum_{S'}(D)$ no pueden ser nulos simultáneamente. Supongamos ahora que seleccionamos S de forma aleatoria: para todo j entre 1 y n , decidimos mediante una moneda imparcial si se incluye o no j en S . Dado que es tan probable que $i \in S$ como que $i \notin S$, la probabilidad de que $\sum_S(D) \neq 0$ es como mínimo de un medio.

Por otra parte, $\sum_S(D)$ siempre es cero si $C = AB$, puesto que en este caso $D = 0$. Esto sugiere comprobar si $C = AB$ o no calculando $\sum_S(D)$ para un subconjunto S seleccionado aleatoriamente, y comparando con 0 el resultado. Pero ¿cómo se puede hacer esto eficientemente sin calcular primero D (y por tanto AB)?

La solución consiste en considerar $\sum_s(D)$ como una multiplicación de matrices independiente. Sea X un vector binario de longitud n tal que $X_i = 1$ si $j \in S$ y $X_i = 0$ en caso contrario. Es sencillo verificar que $\sum_s(D) = XD$, siempre que pensemos en X como en una matriz $1 \times n$. De esta manera, nuestra comprobación se corresponde con verificar si $X(AB - C)$ es o no cero, o equivalentemente si $XAB = XC$, para algún vector binario X seleccionado arbitrariamente. A primera vista, parece que calcular XAB debiera de requerir todavía más tiempo que calcular AB , pero esto no es así. Recordemos de la Sección 8.6 que el tiempo necesario para calcular una multiplicación matricial encadenada depende crucialmente del orden en que se multiplican las matrices. En este caso, sólo se necesita un tiempo que está en $\Theta(n^2)$ para calcular XAB en la forma $(XA)B$, porque éste es el tiempo que se necesita para multiplicar la matriz $1 \times n$ X por la matriz $n \times n$ A , y porque el resultado XC es de nuevo una matriz $1 \times n$ que podemos multiplicar por B en un tiempo que está en $\Theta(n^2)$. Esto sugiere el algoritmo siguiente:

```

función Freivalds( $A, B, C, n$ )
    para  $j \leftarrow 1$  hasta  $n$  hacer  $X_j \leftarrow \text{uniforme}(0..1)$ 
    si  $(XA)B = XC$  entonces devolver verdadero
    sino devolver falso

```

A la vista de lo dicho anteriormente, sabemos que este algoritmo devuelve una respuesta correcta con una probabilidad del 50% en todos los casos, por tanto, es $1/2$ -correcto por definición. Sin embargo no es p -correcto para todo p menor que $1/2$, porque su probabilidad de error es exactamente $1/2$ cuando C difiere de AB precisamente en una fila: se devuelve una respuesta incorrecta si y sólo si $X_i = 0$ donde la fila i supone la diferencia entre C y AB . ¿Es esto realmente interesante? Una probabilidad de error del 50% es intolerable en la práctica, ¿verdad? Hay algo peor: un «algoritmo» más sencillo para alcanzar esta probabilidad de error sería arrojar al aire una moneda imparcial y ¡devolver verdadero o falso dependiendo del resultado, sin siquiera examinar las tres matrices en cuestión! La observación clave es que siempre que $\text{Freivalds}(A, B, C, n)$ devuelve el valor falso, uno puede estar seguro de que la respuesta es correcta: la existencia de un vector X tal que XAB difiere de XC nos permite concluir con certeza que $AB \neq C$. Sólo cuando el algoritmo devuelve verdadero estamos inseguros a la hora de creer su afirmación.

Consideremos por ejemplo las matrices 3×3 siguientes:

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \quad B = \begin{pmatrix} 3 & 1 & 4 \\ 1 & 5 & 9 \\ 2 & 6 & 5 \end{pmatrix} \quad C = \begin{pmatrix} 11 & 29 & 37 \\ 29 & 65 & 91 \\ 47 & 99 & 45 \end{pmatrix}$$

Una llamada a $\text{Freivalds}(A, B, C, 3)$ podría seleccionar $X = (1, 1, 0)$ en cuyo caso XA se obtiene sumando las filas primera y segunda de A , así que $XA = (5, 7, 9)$. Prosiguiendo, calculamos $(XA)B = (40, 94, 128)$. Este resultado se compara con $XC = (40,$

94, 128), que se obtiene sumando la primera y segunda fila de C . Con esta selección de X , *Freivalds* devuelve el valor *verdadero* puesto que es $XAB = XC$. Otra llamada a *Freivalds* podría seleccionar $X = (0, 1, 1)$. En este caso, los cálculos son

$$XA = (11, 13, 15), (XA)B = (76, 166, 236), XC = (76, 164, 136)$$

y *Freivalds* proporciona el valor *falso*. Tenemos más suerte: el hecho de que $XAB \neq XC$ es una demostración concluyente de que $AB \neq C$.

Esto sugiere aplicar *Freivalds*(A, B, C, n) varias veces sobre el mismo caso, empleando tiradas independientes de la moneda en cada ocasión. Si se obtiene la respuesta *falso*, incluso en una sola ocasión, podemos concluir que $AB \neq C$ independientemente del número de veces que se obtenga la respuesta *verdadero*. Consideremos ahora el siguiente algoritmo, en el cual k es un nuevo parámetro.

función *RepetirFreivalds*(A, B, C, n, k)
 para $i \leftarrow 1$ **hasta** k **hacer**
 si *Freivalds*(A, B, C, n) = *falso* **entonces devolver** *falso*
 devolver *verdadero*

Nos interesa la probabilidad de error de este nuevo algoritmo. Consideremos dos casos. Si realmente $AB = C$, entonces toda llamada a *Freivalds* devuelve necesariamente el valor *verdadero*, puesto que no existe la posibilidad de seleccionar aleatoriamente un X tal que $XAB \neq XC$, y por tanto *RepetirFreivalds* devuelve *verdadero* después de pasar k veces por el bucle. En este caso, la probabilidad de error es cero. Por otra parte, si $C \neq AB$, entonces la probabilidad de que una llamada a *Freivalds* devuelva (incorrectamente) *verdadero* es como máximo $1/2$. Estas probabilidades se multiplican, porque los lanzamientos de la moneda son independientes entre llamadas consecutivas. Por tanto, la probabilidad de que k llamadas consecutivas devuelvan todas ellas pueden devolver un valor erróneo de 2^{-k} como máximo. Pero ésta es la única forma en que *RepetirFreivalds* devuelve una respuesta errónea. Concluimos que nuestro nuevo algoritmo $(1 - 2^{-k})$ -correcto. Cuando $k = 10$, esto es mejor que 99,9%-correcto. Repetirlo 20 veces lleva la probabilidad de error a menos de una entre un millón. Este decrecimiento espectacularmente rápido de la probabilidad de error es típico de los algoritmos de Monte Carlo que resuelven problemas de decisión —esto es, problemas para los cuales la respuesta es o bien *verdadero* o bien *falso*— siempre y cuando una de las respuestas, en caso de ser obtenida, tenga garantizada su veracidad.

Alternativamente, se puede dar a los algoritmos de Monte Carlo una cota superior explícita para la probabilidad de error admisible:

función *FreivaldsEpsilon*(A, B, C, n, ϵ)
 $k \leftarrow \lceil \lg 1/\epsilon \rceil$
 devolver *RepetirFreivalds*(A, B, C, n, k)

Una ventaja de esta versión del algoritmo es que podemos analizar su tiempo de ejecución como función simultánea del tamaño del caso y de la probabilidad de error. En este caso, el algoritmo requiere claramente un tiempo que está en

$\Theta(n^2 \log^4 n)$.

Los algoritmos de esta sección tienen un interés práctico limitado, porque se necesitan $3n^2$ multiplicaciones escalares para calcular XAB y XC , en comparación con las n^3 que se necesitan para calcular AB por el método directo. Si insistimos en una probabilidad de error que no sea mayor que una entre un millón, y si de hecho $C = AB$, entonces las 20 ejecuciones necesarias de *Freivalds* efectúan $60n^2$ multiplicaciones escalares, lo cual no es una mejora con respecto a n^3 a no ser que n sea mayor que 60. Sin embargo, esta aproximación podría resultar de utilidad si fuera necesario verificar productos matriciales muy grandes.

10.6.2 Comprobación de primalidad²

Es posible que el algoritmo de Monte Carlo más famoso sea el que decide si un entero impar dado es o no primo. No se conoce ningún algoritmo para resolver este problema con certeza en un tiempo razonable cuando el número que hay que estudiar no tiene más que unos pocos centenares de dígitos decimales. Comprobar la *primalidad* de números grandes es algo más que un *divertimento* matemático: ya vimos en la Sección 7.8 que es crucial para la criptología moderna.

La historia de la comprobación probabilista de *primalidad* tiene sus raíces en Pierre de Fermat, el padre de la Teoría de los Números moderna. En 1640 enunció el teorema siguiente, que a veces se denomina teorema menor de Fermat.

Teorema 10.6.1 (Fermat). Sea n un primo. Entonces $a^{n-1} \bmod n = 1$ para cualquier entero a tal que $1 \leq a \leq n-1$.

Por ejemplo, sean $n = 7$ y $a = 5$. Se tiene que es

$$a^{n-1} = 5^6 = 15625 = 2232 \times 7 + 1$$

y por tanto es $a^{n-1} \bmod n = 1$, tal como debía de suceder. Considérese ahora la versión contrapositiva del teorema de Fermat: si n y a son enteros tales que $1 \leq a \leq n-1$ y si $a^{n-1} \bmod n \neq 1$ entonces n no es primo. Un ejemplo anecdótico lo proporcionó el mismo Fermat. En su búsqueda de una fórmula que sólo produjese números primos, formuló la hipótesis de que $F_n = 2^n + 1$ es primo para todo n . Obsérvese que $F_0 = 3$, $F_1 = 5$, $F_2 = 17$, $F_3 = 257$ y $F_4 = 65537$ son todos ellos números primos. Desafortunadamente, $F_5 = 4294967297$ era un número demasiado grande para que Fermat pudiera intentar demostrar su primalidad. Tuvo que pasar casi un siglo para que Euler factorizase $F_5 = 641 \times 6700417$, probando así la falsedad de la conjetura de Fermat. Irónicamente, Fermat podría haber utilizado su propio teorema para alcanzar la misma conclusión. Estaba dentro de sus posibilidades inventar un algoritmo eficiente de exponenciación modular parecido a *expomod*, y utilizarlo

para calcular $3^{F_5-1} \bmod F_5 = 3029026160 \neq 1$; véase la Sección 7.8. Aunque es tedioso, este cálculo no requiere nada más que 32 cuadrados modulares sucesivos, puesto que $F_5 - 1 = 2^{32}$. A partir de la versión contrapositiva de su propio teorema, Fermat podría haber sabido que F_5 no es primo.

Esto sugiere el siguiente algoritmo probabilista para la comprobación de *primalidad*. Suponemos que $n \geq 2$:

función *Fermat*(n)

$a \leftarrow \text{uniforme}(1..n-1)$

si *expomod*($a, n-1, n$) = 1 **entonces devolver** *verdadero*

sino devolver *falso*

Visto lo anterior, sabemos que n es compuesto cuando una llamada a *Fermat*(n) devuelva el valor *falso* porque la existencia de un a entre 1 y $n-1$ tal que $a^{n-1} \bmod n \neq 1$ basta para demostrar esto, por el teorema menor de Fermat. Es notable que cuando este algoritmo, indica que n es compuesto, no ofrece ni una sola indicación acerca de sus divisores. Teniendo en cuenta los conocimientos algorítmicos actuales, la factorización es mucho más difícil que la comprobación de primalidad. Recuerde que la supuesta diferencia de dificultad entre estos problemas es crucial para el sistema criptográfico RSA descrito en la Sección 7.8: la facilidad de comprobar la primalidad es necesaria para su implementación, mientras que la dificultad de la factorización es condición indispensable para su seguridad.

¿Qué se puede decir, sin embargo, si *Fermat*(n) devuelve el valor *verdadero*? Para concluir que n es primo, necesitaríamos el recíproco y el contrapositivo del teorema de Fermat. Este resultado diría que $a^{n-1} \bmod n$ nunca es igual a 1 cuando n es compuesto y $1 \leq a \leq n-1$. Desafortunadamente, esto no es cierto, porque $1^{n-1} \bmod n = 1$ para todo $n \geq 2$. Además, $(n-1)^{n-1} \bmod n = 1$ para todo $n \geq 3$, porque $(n-1)^2 \bmod n = 1$. El menor contraejemplo no trivial del recíproco es que $4^{14} \bmod 15 = 1$ a pesar de que 15 sea compuesto. Un entero a tal que $2 \leq a \leq n-2$ y que $a^{n-1} \bmod n = 1$ se denomina *testigo falso de primalidad* para n si de hecho n es compuesto. Con tal de modificar la comprobación de Fermat para seleccionar aleatoriamente a entre 2 y $n-2$, sólo fallará en los números compuestos cuando seleccionemos un testigo falso.

Las buenas noticias son que los testigos falsos son bastante escasos. Aunque sólo 5 de entre los 332 impares compuestos menores que 1.000 presumen de no tener falsos testigos, más de la mitad tiene sólo dos testigos falsos y menos del 16% de ellos tiene más de 15. Además, sólo existen 4.490 testigos falsos para todos estos números compuestos juntos. Comparemos esta cantidad con 178.878, que es el número total de testigos candidatos que existe para el mismo conjunto de números. La probabilidad media de error para la comprobación de Fermat aplicada a números impares compuestos menores que 1.000 es menor que 3,3%. Resulta aún menor si consideramos números más grandes.

Las malas noticias son que hay números compuestos que admiten una proporción significativa de falsos testigos. El caso peor entre los números compuestos menores que 1.000 es 561, que admite 318 testigos falsos: son más de la mitad de los testigos po-

tenciales. Un caso más convincente es el que plantea un número de 15 dígitos: *Fermat*(651693055693681) devuelve *verdadero* con una probabilidad mayor que 99,9965% ¡a pesar de que este número es compuesto! Con más generalidad, para cualquier $\delta > 0$ existe un número infinito de números compuestos para los cuales la comprobación de Fermat descubre que son compuestos con una probabilidad menor que δ . En otras palabras, la comprobación de Fermat no es p -correcta para ningún $p > 0$ que fijemos. Consiguientemente, la probabilidad de error no se puede reducir por debajo de un ε arbitrariamente pequeño por repetición de la comprobación de Fermat durante un cierto número prefijado de veces, según la técnica que veíamos en la sección anterior.

Afortunadamente, una pequeña modificación de la comprobación de Fermat resuelve esta dificultad. Sea n un entero impar mayor que 4, y sean s y t enteros tales que $n - 1 = 2^t$, donde t es impar. Obsérvese que $s > 0$ puesto que $n - 1$ es par. Sea $B(n)$ el conjunto formado por estos enteros, definido en la forma siguiente: $a \in B(n)$ si y sólo si $2 \leq a \leq n-2$ y

◇ $a^t \bmod n = 1$, o bien

◇ existe un entero i , $0 \leq i < s$, tal que $a^{2^i} \bmod n = n - 1$

Esto puede parecer complicado, pero es sencillo implementarlo eficientemente. Siempre y cuando n sea impar y $2 \leq a \leq n - 2$, una llamada a *pruebaB*(a , n) devuelve el valor *verdadero* si y sólo si $a \in B(n)$.

función *pruebaB*(a , n)

$s \leftarrow 0$; $t \leftarrow n-1$

repetir

$s \leftarrow s + 1$; $t \leftarrow t \div 2$;

hasta que $t \bmod 2 = 1$

$x \leftarrow \text{expomod}(a, t, n)$

si $x = 1$ **o** $x = n-1$ **entonces devolver** *verdadero*

para $i \leftarrow 1$ **hasta** $s-1$ **hacer**

$x \leftarrow x^2 \bmod n$

si $x = n-1$ **entonces devolver** *verdadero*

devolver *falso*

Por ejemplo, veamos si 158 pertenece a $B(289)$. Hacemos $s = 5$ y $t = 9$ porque $n - 1 = 288 = 2^5 \times 9$. Entonces calculamos

$$x = a^t \bmod n = 158^9 \bmod 289 = 131$$

No concluye aquí la prueba, porque 131 no es 1 es $n - 1 = 288$. A continuación, elevamos x al cuadrado (módulo n) hasta 4 veces ($s-1 = 4$) para ver si obtenemos 288.

$$a^{2^t} \bmod n = 131^2 \bmod 289 = 110$$

$$a^{2^{2^t}} \bmod n = 110^2 \bmod 289 = 251$$

$$a^{2^{2^{2^t}}} \bmod n = 251^2 \bmod 289 = 288$$

Llegados aquí, nos detenemos porque hemos encontrado que es $a^{2^i} \bmod n = n - 1$ para $i=3 < s$, y concluimos que $158 \in B(298)$.

Una extensión del teorema de Fermat demuestra que $a \in B(n)$ para todo $2 \leq a \leq n-2$ cuando n es primo. Por otra parte, se dice que n es un *pseudoprimo fuerte* con base a y que a es un *testigo falso fuerte* de primalidad para n siempre que $n > 4$ es un número impar compuesto y $a \in B(n)$. Por ejemplo, ya hemos visto que 158 es un testigo falso fuerte de primalidad para 289 puesto que $289 = 17^2$ es compuesto. Esto proporciona una comprobación mejor que la de Fermat porque los testigos falsos fuertes son automáticamente testigos falsos respecto a la prueba de Fermat, pero no a la inversa. De hecho, los testigos falsos fuertes son mucho más escasos que los testigos falsos. Por ejemplo, 4 es un testigo falso de primalidad para 15, pero no es un testigo falso fuerte porque $4^7 \bmod 15 = 4$. También vimos que el 561 admite 318 falsos testigos, pero sólo 8 de éstos son testigos falsos fuertes. Considerando todos los impares compuestos menores que 1.000, la probabilidad media de seleccionar aleatoriamente un testigo falso fuerte es menor que el 1%; más del 72% de estos números compuestos ni siquiera admite un único falso testigo fuerte. La superioridad de la prueba fuerte queda ilustrada de la mejor manera posible por el hecho consistente en que todos los enteros compuestos entre 5 y 10^{13} no son pseudoprimos fuertes en al menos una de las bases 2, 3, 5, 7 o 61. En otras palabras, cinco llamadas a *pruebaB* bastan para decidir de *forma determinista* acerca de la primalidad de cualquier entero hasta 10^{13} . Hay algo más importante: a diferencia de la comprobación de Fermat, se garantiza que la proporción de testigos falsos fuertes es pequeña *para todo número impar compuesto*. Más precisamente, tenemos el teorema siguiente.

Teorema 10.6.2. Consideremos un número impar arbitrario $n > 4$:

◇ Si n es primo, entonces $B(n) = \{a \mid 2 \leq a \leq n-2\}$

◇ Si n es compuesto, entonces $|B(n)| \leq (n-9)/4$

Consiguientemente, *pruebaB*(a, n) siempre devuelve el valor *verdadero* cuando n es un primo mayor que 4, y $2 \leq a \leq n-2$, mientras que devuelve el valor *falso* con una probabilidad de más de 3/4 cuando n es un entero impar compuesto mayor que 4 y a se selecciona aleatoriamente entre 2 y $n-2$. En otras palabras, el siguiente algoritmo de Monte Carlo es 3/4-correcto para la comprobación de primalidad; se conoce con el nombre de comprobación de Miller-Rabin:

función *MillRab*(n)

{Este algoritmo sólo debe invocarse si $n > 4$ es impar}

$a \leftarrow \text{uniforme}(2..n-2)$

devolver *pruebaB*(a, n)

Dado que se garantiza que la respuesta *falso* sea correcta, la técnica que vimos en la sección anterior es aplicable para reducir muy rápidamente la probabilidad de error.

función *RepetirMillRab*(n, k)
 {Este algoritmo sólo debe invocarse si $n > 4$ es impar}
para $i \leftarrow 1$ **hasta** k **hacer**
 si *MillRab*(n) = falso **entonces devolver** falso
devolver verdadero

Este algoritmo siempre devuelve la respuesta correcta cuando $n > 4$ es primo. Cuando $n > 4$ es un impar compuesto, cada llamada a *MillRab* tiene como mucho una probabilidad de $1/4$ de llegar a un testigo falso fuerte, devolviendo *verdadero* de manera errónea. Dado que la única manera de que *RepetirMillRab* devuelva el valor *verdadero* en este caso es encontrar aleatoriamente k testigos falsos fuertes seguidos, esto sucede con una probabilidad que es como mucho de 4^{-k} . Por ejemplo, basta con tomar $k = 10$ para reducir la probabilidad a menos de una entre un millón. En conclusión, *RepetirMillRab*(n, k) es un algoritmo de Monte Carlo $(1 - 4^{-k})$ -correcto para la comprobación de primalidad.

¿Cuánto tiempo se necesita para decidir la primalidad de n con una probabilidad de error acotada por ε ? Debemos repetir la comprobación de Miller-Rabin k veces, en donde es $4^{-k} \leq \varepsilon$, que equivale a $2^{2k} \geq 1/\varepsilon$. Esto se consigue mediante $k = \lceil \frac{1}{2} \lg \frac{1}{\varepsilon} \rceil$. Cada llamada a *MillRab* requiere una exponenciación modular, con t como exponente, y $s-1$ elevaciones al cuadrado modulares. Sabemos por la Sección 7.8 que la exponenciación requiere un número de multiplicaciones y elevaciones al cuadrado modulares que está en $O(\log t)$. Contando las elevaciones al cuadrado como multiplicaciones, y dado que es $\lg n > \lg(n-1) = \lg(2^s) t = s + \lg t$, el tiempo requerido por una llamada a *MillRab* está dominado por un número de multiplicaciones modulares que está en $O(\log n)$. Si se efectúan éstas según el algoritmo clásico, cada una de ellas requiere un tiempo que está en $O(\log^2 n)$ porque reducimos módulo n después de cada multiplicación.

Resumiendo, el tiempo total necesario para decidir acerca de la primalidad de n con una probabilidad de error acotada por ε está por tanto en $O(\log^3 n \log \frac{1}{\varepsilon})$. Esto es enteramente razonable en la práctica para números de mil dígitos y una probabilidad de error menor que 10^{-100} .

10.6.3 ¿Puede un número ser probablemente primo?

Supongamos que se aplica diez veces la comprobación de Miller-Rabin a algún entero impar n , y que se obtiene la respuesta *verdadero* en las diez ocasiones. ¿Qué podemos deducir? Hemos visto que si n era compuesto, esto podría suceder con una probabilidad máxima de $4^{-10} = 2^{-20}$, que es menor que uno entre un millón. Puede tentarnos concluir que « n es primo, salvo por una probabilidad de error dada por 2^{-20} ». Sin embargo, esta afirmación carece de sentido: ¡todo entero mayor que 1 es o bien primo o bien compuesto! Lo mejor que podemos decir es «creo que n es primo; en caso contrario he observado un fenómeno cuya probabilidad de ocurrir era menor que uno entre un millón». Además, hasta esta afirmación prudencial debe considerarse con cautela si los testigos de la comprobación se han obtenido con un generador pseudoaleatorio, en lugar de ser realmente alea-

torios: quizá n sea compuesto y no hemos conseguido hallar un testigo, pero no por tener mala suerte sino porque el generador pseudoaleatorio era defectuoso; véase la Sección 10.4. Sin embargo, este fallo es improbable si se emplea un buen generador, y no nos preocuparemos más acerca de esta posibilidad.

Dado que es imposible llegar a estar seguros de que un número dado es primo mediante este algoritmo de Monte Carlo —sólo se pueden certificar los números compuestos— y dado que los primos son relevantes a efectos criptográficos, quizá se pregunte si merece la pena arriesgarse a obtener un «primo falso» para ahorrar tiempo de cálculo. Si su vida dependiera de la primalidad de un número dado, ¿no invertiría el tiempo necesario para certificar su primalidad empleando el mejor método determinista? Digamos que se tiene que decidir entre ejecutar *RepetirMillRab*(n , 150) o emplear algún método más sofisticado, que requiere un tiempo significativamente mayor pero garantiza una respuesta correcta. ¿Cuál de estas aproximaciones tiene menos probabilidades de inducirnos a error? Tal como se discute en la Sección 10.2, el método sofisticado tiene más probabilidades de ser incorrecto, a pesar de sus resultados, porque la probabilidad de que se produzca un error de *hardware* no detectado durante su (más extenso) periodo de cálculo puede ser mayor que $4^{-150} \approx 10^{-100}$. Además, uno puede tener más confianza en que su implementación de la comprobación de Miller-Rabin esté libre de errores que en el algoritmo más complicado. Así llegamos a la paradójica conclusión consistente en que un primo probable, sea cual fuere el significado dado a esta frase, ¡puede ser más fiable que uno cuya primalidad se haya «demostrado» de forma determinista! Obsérvese sin embargo que existen métodos probabilistas que producen primos aleatorios *certificados* en un tiempo menor que el que se necesita para hallar primos probabilistas mediante el uso repetido de la prueba de Miller-Rabin. Dado que se garantiza que estos métodos van a proporcionar un número primo, pertenecen a la clase de algoritmos de Las Vegas.

La importancia de interpretar correctamente el resultado de un algoritmo de Monte Carlo se puede ilustrar muy bien si se desea *generar* un número de ℓ dígitos que sea probablemente primo, quizá con propósitos criptográficos. El algoritmo evidente consiste en seguir seleccionando enteros impares aleatorios de ℓ dígitos hasta encontrar uno que pase por un número suficiente de aplicaciones de la comprobación de Miller-Rabin. Más exactamente, considérese el siguiente algoritmo, que sólo debería de utilizarse con $\ell > 1$:

función *primoaleatorio*(ℓ , k)

repetir

{Seleccionar un n impar aleatoriamente entre $10^{\ell-1}$ y $10^\ell - 1$ }

$n \rightarrow 1 + 2 \times \text{uniforme}(10^{\ell-1} / 2..10^\ell / 2 - 1)$

hasta que *RepetirMillRab*(n , k)

devolver n

¿Qué se puede decir acerca del resultado de este algoritmo? Ya sabemos que no se puede afirmar que el número obtenido sea primo con una probabilidad de al menos $1 - 4^{-k}$. Sin embargo, tiene sentido investigar el número medio de «números primos falsos» (un eufemismo de «números compuestos») que se pro-

ducen al ejecutar m veces el algoritmo. Resulta tentador concluir de la discusión anterior que la respuesta es «como mucho $4^{-k}m$ porque nuestra probabilidad de error en cada uno de los m números producidos es como máximo de 4^{-k} ». Por ejemplo, si llamamos a *primoaleatorio*(1.000, 5) un millón de veces, esperaríamos menos de 1.000 números compuestos entre el millón de números de 1.000 cifras producidos así (porque es $4^{-5}10^6 \approx 997$). Bastaría con llamar a *primoaleatorio*(1.000, 10) en su lugar para reducir este número esperado a menos de 1, a costa de no llegar a doblar el tiempo de ejecución. Esta conclusión es correcta, pero sólo porque la probabilidad de error esperada para la comprobación de Miller-Rabin para un número impar compuesto seleccionado aleatoriamente es mucho menor que $1/4$. Sin embargo, ¡el razonamiento simplificado es incorrecto!

El problema es que un número impar aleatorio muy grande tiene muchas más probabilidades de ser compuesto que de ser primo. Por tanto, una llamada a *primoaleatorio* probablemente utilizará *RepetirMillRab* para probar muchos números compuestos antes de comprobar un número primo, si esto llega a suceder. La probabilidad de error inherente en cada llamada a *RepetirMillRab* aplicándolo a números compuestos se irá acumulando. Aun cuando todas las llamadas tienen probabilidad de encontrar que su número es compuesto, la probabilidad de que una de las llamadas se equivoque —dando lugar a que *primoaleatorio* devuelva este número compuesto por error— no es despreciable si son muchos los números compuestos que se comprueban antes de hallar un número primo por casualidad. Consideremos lo que sucedería si la probabilidad de error de la prueba de Miller-Rabin fuera exactamente $1/4$ en todos los compuestos impares, y si llamáramos a *primoaleatorio*(1.000, 5). En cada pasada por el bucle *repetir*, sucede una de estas tres cosas:

- ◇ Si el número n seleccionado aleatoriamente es primo, entonces *RepetirMillRab*(n , 5) devolverá *verdadero* con toda certeza, finalizando así el bucle; *primoaleatorio* devuelve correctamente un número primo en este caso.
- ◇ Si el número n seleccionado aleatoriamente es compuesto, entonces hay una probabilidad exactamente igual a $4^{-5} = 1/1024$ de que *RepetirMillRab*(n , 5) devuelva el valor *verdadero*, finalizando el bucle; *primoaleatorio* devuelve erróneamente un número compuesto en este caso.
- ◇ En caso contrario, el algoritmo vuelve al principio del bucle con la selección aleatoria de otro número impar de mil cifras.

La probabilidad de que un número impar aleatorio de mil cifras sea primo es aproximadamente de uno entre mil; véase el problema 10.19. Por tanto, los dos primeros casos son casi equiprobables, mientras que el tercero es con mucho el más probable. Como consecuencia, el bucle de *primoaleatorio* tiene las mismas probabilidades de acabar por cualquiera de las razones posibles, así que ¡devuelve números compuestos casi con tanta frecuencia como números primos! (Una

10.6.4 Amplificación de la ventaja estocástica

Supongamos que disponemos de un algoritmo de Monte Carlo *no* sesgado cuya probabilidad de error sea no nula independientemente del caso que haya que resolver, y de la respuesta proporcionada. ¿Sigue siendo posible reducir arbitrariamente la probabilidad de error repitiendo el algoritmo? La respuesta es que depende de la probabilidad de error original. Por sencillez, vamos a concentrarnos en algoritmos que conciernen a la toma de decisiones; véase sin embargo el problema 10.22). Considere un algoritmo de Monte Carlo del cual lo único que se sabe es que es p -correcto. El primer comentario evidente es que la amplificación de la ventaja estocástica es imposible a no ser que $p > 1/2$ porque siempre existe el (inútil) algoritmo $1/2$ -correcto:

Sea MC un algoritmo de Monte Carlo no sesgado 3/4-correcto para resolver algún problema de decisión. Considérese el siguiente algoritmo, que llama tres veces a $MC(x)$ y devuelve la respuesta más frecuente:

```

función MC3(x)
   $t \leftarrow MC(x); u \leftarrow MC(x); v \leftarrow MC(x);$ 
  si  $t = u$  o  $t = v$  entonces devolver  $t$ 
  sino devolver  $u$ 

```

¿Cuál es la probabilidad de error de *MC3*? Sean *R* y *W* las respuestas correcta e incorrecta respectivamente. Sabemos que *t*, *u* y *v* tienen una probabilidad mínima de $\frac{3}{4}$ de ser *R*, independientemente unas de otras. Supongamos por sencillez que esta probabilidad es exactamente de $\frac{3}{4}$, puesto que claramente el algoritmo *MC3* sería todavía mejor si la probabilidad de error de *MC* fuera menor que $\frac{1}{4}$. Hay ocho resultados posibles para las tres llamadas a *MC*, cuyas probabilidades se resumen en la tabla siguiente:

<i>t</i>	<i>u</i>	<i>v</i>	prob	<i>MC3</i>
R	R	R	27/64	R
R	R	W	9/64	R
R	W	R	9/64	R
R	W	W	3/64	W
W	R	R	9/64	R
W	R	W	3/64	W
W	W	R	3/64	W
W	W	W	1/64	W

Sumando las probabilidades asociadas a las filas 1, 2, 3 y 5 concluimos que *MC3* es correcto con una probabilidad de 27/32, que es mejor que 84%.

Más generalmente, sea *MC* un algoritmo de Monte Carlo para resolver algún problema de decisión cuya ventaja sea $\epsilon > 0$. Consideremos el siguiente algoritmo, que llama *k* veces a *MC* y devuelve la respuesta más frecuente:

función *RepetirMC*(*x*, *k*)

$V, F \leftarrow 0$

para *i* $\leftarrow 1$ **hasta** *k* **hacer**

si *MC*(*x*) **entonces** $V \leftarrow V + 1$

sino $F \leftarrow F + 1$

si $V > F$ **entonces devolver** *verdadero* **sino devolver** *falso*

¿Cuál es la probabilidad de error de *RepetirMC*? Para averiguarlo, asociamos una variable aleatoria *X_i* a cada llamada a *MC*: $X_i = 1$ si se obtiene la respuesta correcta y $X_i = 0$ en caso contrario. Por suposición, $\Pr[X_i = 1] \geq \frac{1}{2} + \epsilon$ para todo *i*. Supongamos por sencillez que esta probabilidad es exactamente de $\frac{1}{2} + \epsilon$ puesto que en otro caso *RepetirMC* sería incluso mejor. Suponga también que *k* es impar, para evitar el riesgo de un empate ($V = F$) en la votación de mayoría; véase el problema 10.23. La esperanza y la varianza de *X_i* son $E(X_i) = \frac{1}{2} + \epsilon$ y $\text{Var}(X_i) = (\frac{1}{2} + \epsilon)(\frac{1}{2} - \epsilon) = \frac{1}{4} - \epsilon^2$, respectivamente. Ahora, sea $X = \sum_{i=1}^k X_i$ la variable aleatoria que corresponde al número de respuestas correctas al cabo de *k* intentos. Para cualquier entero *i* entre 0 y *k*

$$\Pr[X = i] = \binom{k}{i} \left(\frac{1}{2} + \varepsilon\right)^i \left(\frac{1}{2} - \varepsilon\right)^{k-i} \quad (10.2)$$

y X tiene la esperanza $E(X) = (1/2 + \varepsilon)k$ y la varianza $\text{Var}(x) = (1/4 - \varepsilon^2)k$. Nos interesa $\Pr[X \leq k/2]$, que es la probabilidad de error de *RepetirMC*.

Dados unos valores específicos para ε y k , podemos calcular esta probabilidad mediante el algoritmo de programación dinámica dado en la Sección 8.1.2 para el campeonato mundial, o bien podemos calcularlo en la forma $\sum_{i=0}^{k/2} \Pr[X=i]$ empleando la ecuación 10.2. Mejor aún, podemos emplear la fórmula del problema 10.25. Sin embargo, resulta más cómodo utilizar el Teorema Central del Límite, que dice que la distribución de X es casi normal cuando k es grande; en la práctica, $k = 30$ ya es bastante grande. Supongamos por ejemplo que se desea una probabilidad de error menor que el 5%. Las tablas para la distribución normal nos dicen que

$$\Pr[X < E(X) - 1.645 \sqrt{\text{Var}(X)}] \approx 5\%.$$

Para obtener $\Pr[X \geq k/2] < 5\%$ basta que

$$k/2 < E(X) - 1.645 \sqrt{\text{Var}(X)}.$$

Empleando nuestros valores para $E(X)$ y $\text{Var}(X)$, esta condición se traduce en que

$$k > 2,706 \left(\frac{1}{4\varepsilon^2} - 1 \right). \quad (10.3)$$

Por ejemplo, si $\varepsilon = 5\%$, la ecuación 10.3 nos dice que «basta» con ejecutar 269 veces un algoritmo de Monte Carlo no sesgado 55%-correcto y tomar la respuesta más frecuente para obtener un algoritmo 95%-correcto. (La ecuación 10.3 da $k > 267,894$ pero hemos tomado $k = 269$ en lugar de $k = 268$ porque deseábamos que k fuera impar; véase el problema 10.23). En otras palabras, esta repetición traduce una ventaja de un 5% en una probabilidad de error de un 5%. Un cálculo exacto muestra que la probabilidad de error resultante está justo por encima del 4,99%, ilustrando la precisión de la aproximación normal para valores tan grandes de k . Esto muestra el inconveniente de los algoritmos de Monte Carlo no sesgados: ejecutar un algoritmo *sesgado* 55%-correcto sólo 4 veces reduce la probabilidad de error a un 4,1%, puesto que $0,45^4 \approx 0.041$.

La ecuación 10.3 muestra que el número de repeticiones necesario para alcanzar un nivel de confianza dado (el 95% en este caso) depende fuertemente de la ventaja ε del algoritmo original. Una ventaja 10 veces menor requeriría aproximadamente 100 veces más repeticiones para obtener la misma fiabilidad. Por otra parte, no resulta mucho más oneroso obtener una confianza bastante mayor en la respuesta final. Por ejemplo, si hubiésemos deseado una probabilidad 10 veces más pequeña, habríamos utilizado

$$\Pr[X < E(X) - 2,576 \sqrt{\text{Var}(\bar{X})}] \approx \frac{1}{2}\%$$

para concluir que basta con repetir un algoritmo de Monte Carlo no sesgado $(1/2 + \varepsilon)$ -correcto un número de veces $6,636\left(\frac{1}{\varepsilon^2} - 1\right)$ para hacer que sea 99,5%-correcto. Esto no es ni siquiera 2,5 veces más costoso que lograr una corrección del 95%. Existe un argumento de combinatoria que muestra que para llevar la probabilidad de error por debajo de δ para un algoritmo de Monte Carlo no sesgado cuya ventaja sea ε , el número de repeticiones necesario es proporcional a $1/\varepsilon^2$, tal como se vio, pero también a $\log 1/\delta$. Véanse los problemas 10.24 y 10.25 para más detalles.

10.7 ALGORITMOS DE LAS VEGAS

Los algoritmos de Las Vegas toman decisiones probabilistas como ayuda para guiarse más rápidamente hacia una solución correcta. A diferencia de los algoritmos de Monte Carlo, nunca proporcionan una respuesta incorrecta. Hay dos categorías principales de algoritmos de Las Vegas. Pueden utilizar la aleatoriedad para guiar su búsqueda de tal manera que se garantice una solución correcta aun cuando se tomen decisiones poco afortunadas: si esto ocurriera, tan sólo necesitarán más tiempo. Alternativamente, pueden permitirse a sí mismos tomar caminos equivocados, que los llevarán a un callejón sin salida, haciendo imposible hallar una solución en esta ejecución del algoritmo.

Los algoritmos de Las Vegas del primer tipo suelen utilizarse cuando un algoritmo determinista conocido para resolver el problema en cuestión es mucho más rápido en el caso promedio que en el caso peor. *Quicksort*, de la Sección 7.4.2, ofrece el ejemplo más famoso; véase la Sección 10.7.2. Al incorporar un elemento aleatorio, se permite a los algoritmos de Las Vegas reducir, y a veces eliminar, esta diferencia entre casos buenos y malos. No es cuestión de evitar la aparición ocasional del comportamiento del algoritmo en el caso peor, sino más bien de romper la conexión entre la aparición de este comportamiento y el caso concreto que haya que resolver.

Recordemos de la Sección 2.4 que el análisis de la eficiencia en el caso del promedio de un algoritmo puede a veces producir resultados que induzcan a confusión. La razón es que todo análisis del caso medio debe estar basado en una hipótesis acerca de la distribución de probabilidad de los casos que haya que manejar. Una hipótesis que sea correcta para una aplicación dada del algoritmo puede resultar desastrosamente incorrecta para otra aplicación diferente. Supongamos, por ejemplo, que se utiliza *quicksort* como subalgoritmo dentro de un algoritmo más complejo. Vimos en la Sección 7.4.2 que requiere un tiempo que está en $\Theta(n \log n)$ para ordenar n elementos siempre y cuando los casos que haya que ordenar se seleccionen de forma aleatoria. Este análisis ya no tiene relación alguna con la realidad si de hecho entregamos al algoritmo casos que ya estén casi ordenados. En general, estos algoritmos deterministas son vulnerables a una distribución de probabilidad inesperada de los ca-

sos que algunas aplicaciones pueden pasarles para su resolución: incluso cuando las circunstancias del caso peor sean muy raras, podrían ser las más relevantes para la aplicación, dando lugar a una espectacular degradación del rendimiento. Los algoritmos de Las Vegas nos liberan de preocupaciones acerca de estas situaciones, igualando el tiempo necesario para diferentes casos de un tamaño dado.

El rendimiento de estos algoritmos de Las Vegas no es mejor que el del algoritmo determinista correspondiente cuando consideramos la media para todos los casos de un tamaño dado. Con mucha probabilidad, los casos que requirieran mucho tiempo con el método determinista se resuelven ahora mucho más deprisa, pero los casos en que el algoritmo determinista fuera especialmente bueno se ralentizan hasta un valor medio en el algoritmo de Las Vegas. De esta manera, estos algoritmos de Las Vegas «roban» tiempo de los casos «ricos» (los que resolvía rápidamente el algoritmo determinista) para dárselo a los casos «pobres». Llamamos a esto el *efecto Robin Hood*, y lo ilustramos en las Secciones 10.7.2 y 10.7.3. Esto es interesante cuando consideramos algoritmos deterministas que no son significativamente más rápidos que en el caso medio en el mejor de los casos, así que no se pierde gran cosa al reducirlos a la media, pero que sufren de unos pocos casos malos especialmente graves.

La otra categoría de algoritmos de Las Vegas se caracteriza por tomar de vez en cuando decisiones que los llevan a un callejón sin salida. Se pide que estos algoritmos sean capaces de reconocer su situación, en cuyo caso se limitarán a admitir el fallo. Este comportamiento sería intolerable en un algoritmo determinista, por cuanto supondría que no sería capaz de resolver el caso considerado. Sin embargo, la naturaleza probabilista de los algoritmos de Las Vegas hace que esta admisión de fallo sea aceptable siempre y cuando no se produzca con excesiva probabilidad: cuando se produzca un fracaso basta con volver a aplicar el mismo algoritmo al mismo caso para tener una nueva posibilidad de éxito. Hay problemas prácticos para los cuales esta disposición a arriesgarse a un fallo permite un algoritmo de Las Vegas eficiente, cuando no se conocen algoritmos deterministas que sean eficientes; véase la Sección 10.7.4. Aunque no se puede establecer una cota superior garantizada para el tiempo que se necesitará para obtener una respuesta si se reinicia el algoritmo siempre que falle, este tiempo puede ser razonable con una elevada probabilidad. Estos algoritmos no debieran de confundirse con aquellos que, como el algoritmo *simplex* para programación lineal, son extremadamente eficientes para la gran mayoría de los casos que hay que manejar, pero catastróficos para unos pocos casos: un algoritmo de Las Vegas deberá de tener un buen rendimiento esperado, sea cual sea el caso que haya que resolver.

Cuando se permite que falle un algoritmo de Las Vegas, es más cómodo representarlo en la forma de un **procedimiento** que en forma de una **función**. Esto permite tener un parámetro de salida *éxito*, que recibe el valor *verdadero* si se obtiene una solución, o *falso* en caso contrario. La llamada típica para resolver el caso x es $LV(x, y, \text{éxito})$, en donde el parámetro de salida y recibe la

solución siempre que *éxito* reciba el valor *verdadero*. Por comodidad, escribiremos

devolver *éxito* $\leftarrow$ *verdadero*

como equivalente de

éxito $\leftarrow$ *verdadero*
devolver *éxito*

y el equivalente para **devolver** *éxito* $\leftarrow$ *falso*.

Sea $p(x)$ la probabilidad de éxito del algoritmo cada vez que se le pide que resuelva el caso x . Para que un algoritmo merezca el nombre «Las Vegas», exigimos que $p(x) > 0$ para todo caso x . Esto asegura que se encontrará una solución eventualmente si seguimos repitiendo el algoritmo. Es aún mejor si existe una constante $\delta > 0$ tal que $p(x) \geq \delta$ para todo caso x , puesto que en caso contrario el número esperado de repeticiones antes del éxito podría crecer arbitrariamente con el tamaño del caso.

Consideremos el algoritmo siguiente:

función *RepetirLV*(x)
 repetir
 $LV(x, y, \text{éxito})$
 hasta que *éxito*
 devolver y

Dado que cada llamada a $LV(x)$ tiene una probabilidad de éxito $p(x)$, el número esperado de pasadas por el bucle es $1/p(x)$. Un parámetro más interesante es el tiempo esperado $t(x)$ antes de que *RepetirLV*(x) tenga éxito. Puede uno pensar a primera vista que $t(x)$ es simplemente $1/p(x)$ multiplicado por el tiempo esperado que requiera cada llamada a $LV(x)$. Sin embargo, un análisis correcto debe considerar por separado el tiempo esperado tomado por $LV(x)$ en caso de éxito y en caso de fracaso. Denotemos estos tiempos esperados mediante $s(x)$ y $f(x)$, respectivamente. Despreciando el tiempo que necesita el control del bucle **repetir**, $t(x)$ viene dado por un análisis de casos:

Con probabilidad $p(x)$, la primera llamada a $LV(x)$ tiene éxito al cabo de un tiempo esperado $s(x)$.

Con probabilidad $1 - p(x)$, la primera llamada a $LV(x)$ fracasa al cabo de un tiempo esperado $f(x)$. Después, volvemos al punto de partida, y seguimos estando a un tiempo esperado $t(x)$ del éxito. El tiempo esperado total en este caso es por tanto $f(x) + t(x)$.

Por tanto, $t(x)$ viene dado por una sencilla recurrencia

$$t(x) = p(x)s(x) + (1-p(x))(f(x) + t(x)),$$

que se resuelve fácilmente para dar

$$t(x) = s(x) + \frac{1-p(x)}{p(x)} f(x) \quad (10.4)$$

Algunos algoritmos de Las Vegas permiten el ajuste fino de varios parámetros. Al girar un mando, por ejemplo, decrece tanto el tiempo esperado en el caso de éxito o de fracaso (lo cual es bueno) como la probabilidad de éxito (lo cual es malo). Cuando sucede esto, la ecuación 10.4 es la clave para optimizar el rendimiento global del algoritmo. Ilustramos esto en la Sección 10.7.1 donde volvemos a tratar al problema de las 8 reinas. La Sección 10.7.4 ofrece un ejemplo más útil y sofisticado de algoritmo de Las Vegas que puede fallar en ocasiones, pero para el cual hay una estrategia mejor que reiniciar todo el cálculo en caso de fracasar.

10.7.1 El problema de las ocho reinas, segunda parte

El problema de las ocho reinas proporciona un ejemplo instructivo de un algoritmo de Las Vegas que se beneficia de permitirle fallar. Recordemos que la técnica de retroceso empleada en la Sección 9.6.2 requiere explorar sistemáticamente los nodos del árbol implícito formado por los llamados vectores k -prometedores. Empleando esta técnica, obtenemos la primera solución después de examinar solamente 114 de los 2.057 nodos que hay en el árbol. Esto no está mal, pero el algoritmo no tiene en cuenta un hecho importante: en la mayoría de las soluciones, nada tienen de sistemático las posiciones de las reinas. Por el contrario, las reinas parecen haber sido situadas al azar.

Esta observación sugiere un algoritmo voraz de Las Vegas que vaya poniendo aleatoriamente reinas en filas sucesivas, ocupándose tan sólo de no poner nunca una nueva reina en una posición que sea amenazada por otra reina ya situada. Dado que éste es un algoritmo voraz, no se hace intento alguno de reubicar las reinas anteriores cuando ya no hay posibilidades para la reina que falta: estamos en un callejón sin salida. Como resultado, el algoritmo o bien acaba con éxito si se las arregla para situar todas las reinas en el tablero, o bien fracasa si no hay ningún cuadrado en el que se pueda añadir la reina siguiente. La aproximación voraz hace el algoritmo más sencillo de entender que el de vuelta atrás, y también más fácil de implementar a mano —aun cuando el código sea más largo— pero también implica una posibilidad de fracaso. ¿Se obtiene esta sencillez a costa de la eficiencia? Veremos que nada podría estar más lejos de la verdad. El algoritmo emplea los mismos conjuntos *col*, *diag45* y *diag135* que en la Sección 9.6.2 como ayuda para determinar qué posiciones siguen estando disponibles en la fila estudiada:

```

procedimiento reinasLV(var sol[1..8], éxito)
  vector ok[1..8] {contendrá las posiciones disponibles}
  col, diag45, diag135  $\leftarrow \emptyset$ 
  para k  $\leftarrow 0$  hasta 7 hacer
    {sol[1..k] es k-prometedor; situemos la reina (k+1)-ésima}
    nb  $\leftarrow 0$  {para contar el número de posibilidades}
    para j  $\leftarrow 1$  hasta 8 hacer
      si  $j \notin col$  y  $j-k \notin diag45$  y  $j+k \notin diag135$ 
        entonces {la columna j está disponible para la (k+1)-ésima reina}
          nb  $\rightarrow nb+1$ 
          ok[nb]  $\leftarrow j$ 
    si nb = 0 entonces terminar éxito  $\leftarrow falso$ 
    j  $\leftarrow ok[uniforme(1..nb)]$ 
    col  $\leftarrow col \cup \{j\}$ 
    diag45  $\leftarrow diag45 \cup \{j-k\}$ 
    diag135  $\leftarrow diag135 \cup \{j+k\}$ 
    sol[k+1]  $\leftarrow j$ 
  {fin del bucle para en k}
devolver éxito  $\leftarrow verdadero$ 

```

Para analizar este algoritmo, necesitamos determinar su probabilidad de éxito p , el número medio s de nodos que explora en caso de éxito, y el número medio f de nodos que explora en caso de fracaso. Claramente, $s = 9$, contando el vector vacío 0-prometedor y la solución 8-prometedora. Empleando una computadora, podemos calcular $p \approx 0,1293$ y $f \approx 6,971$. Por tanto, ¿se obtiene una solución más de una vez de entre ocho sin más que proceder de forma totalmente aleatoria! El número esperado de nodos explorados si repetimos el algoritmo hasta por alcanzar el éxito está dado por la ecuación 10.4: $s + \frac{1-p}{p} f \approx 55,93$, que es menos de la mitad del número de nodos explorados por vuelta atrás sistemática.

Podemos hacerlo aún mejor. El algoritmo de Las Vegas es demasiado derrotista: en cuanto encuentra un fallo, vuelve a empezar otra vez desde el principio. El algoritmo de vuelta atrás, por otra parte, hace una búsqueda sistemática de una solución que no tiene nada de sistemática, como sabemos. Una combinación juiciosa de ambos algoritmos sitúa primero aleatoriamente un cierto número de reinas en el tablero, y después utiliza la vuelta atrás para intentar añadir las reinas restantes, sin reconsiderar, sin embargo, las posiciones de las reinas que se hayan colocado aleatoriamente.

Una colocación aleatoria desafortunada de las posiciones de las primeras reinas puede hacer imposible añadir todas las demás. Esto sucede, por ejemplo, si las dos primeras reinas se ponen respectivamente en las posiciones 1 y 3. Cuantas mas reinas situemos aleatoriamente, menor será el tiempo medio que necesita la fase subsiguiente de vuelta atrás, tanto si tiene éxito como si falla, pero mayor será la probabilidad de fallo. Éste es el «mando de ajuste fino» que se mencionaba anteriormente. Hagamos que *detenerLV* denote el número de

reinas que se colocan aleatoriamente antes de pasar a la fase de vuelta atrás con $0 \leq \text{detenerLV} \leq 8$.

El algoritmo modificado es similar a *reinasLV* salvo que debemos incluir la declaración de un **procedimiento** *vueltaatrás* interno (véase más abajo), el bucle en *k* debe ir desde 0 hasta *detenerLV*–1, y sustituiremos la última línea (**terminar** *éxito* $\leftarrow$ *verdadero*) por

vueltaatrás(*detenerLV*, *col*, *diag45*, *diag135*, *éxito*)

Esta instrucción invoca a la fase de vuelta atrás siempre y cuando el bucle no haya terminado prematuramente por un fallo. El **procedimiento** *vueltaatrás* se parece al algoritmo *reinas* de la Sección 9.6.2, salvo que tiene el parámetro adicional *éxito* y que termina inmediatamente después de hallar la primera solución, o después de determinar que no hay ninguna, según sea el caso.

Para dar al mando de ajuste fino su posición óptima, necesitamos determinar la probabilidad de éxito *p*, el número esperado *s* de nodos explorados en caso de éxito y el número esperado *f* de nodos explorados en caso de fracaso para cada valor posible de *detenerLV*. Entonces se puede utilizar la ecuación 10.4 para determinar el número esperado *t* de nodos explorados si se repite el algoritmo hasta que eventualmente encuentre una solución. Estos números, que se obtienen explorando todo el árbol de vuelta atrás con ayuda de una computadora, se resumen en la figura 10.3. El caso *detenerLV* = 0 corresponde a utilizar directamente el algoritmo determinista.

<i>detenerLV</i>	<i>p</i>	<i>s</i>	<i>f</i>	<i>t</i>
0	1,0000	114,00	—	114,00
1	1,0000	39,63	—	39,63
2	0,8750	22,53	39,67	28,20
3	0,4931	13,48	15,10	29,01
4	0,2618	10,31	8,79	35,10
5	0,1624	9,33	7,29	46,92
6	0,1357	9,05	6,98	53,50
7	0,1293	9,00	6,97	55,93
8	0,1293	9,00	6,97	55,93

Figura 10.3. Ajuste fino de un algoritmo de Las Vegas

Aunque una aproximación puramente probabilista (*detenerLV* = 8) es mejor que el determinismo puro (*detenerLV* = 0), una mezcla de ambas es mejor todavía. El compromiso más grato cuando se ejecuta a mano el algoritmo consiste en poner aleatoriamente las tres primeras reinas (*detenerLV* = 3) y seguir por vuelta atrás: esencialmente, esto falla en la mitad de las ocasiones, pero es muy rápido tanto si tiene éxito como si no. ¡Pruébalo, es divertido!

El número de nodos explorados es una buena medida de la cantidad de trabajo efectuado por el algoritmo, pero puede no dar una imagen completa de la situación.

Para asegurarnos, implementamos el algoritmo *reinasLV* en una estación de trabajo. La vuelta atrás pura encuentra la solución en 0,45 milisegundos, mientras que basta una media de 0,14 milisegundos si se sitúan dos reinas al azar antes de volver atrás. Esto es más de tres veces más rápido. Sin embargo, se necesitan 0,21 milisegundos de análisis en el caso medio si se sitúan al azar las tres primeras reinas, lo cual sólo es el doble de bueno que la vuelta atrás pura. Si se colocan todas las reinas de modo aleatorio, perdemos: se necesita análisis en el caso medio de casi un milisegundo para hallar una solución, que es casi el doble del tiempo necesario de forma determinista. Este resultado decepcionante se explica fácilmente: el tiempo necesario para generar valores pseudoaleatorios no se puede despreciar. Resulta que el 71% del tiempo invertido en resolver el problema de las ocho reinas cuando se sitúan aleatoriamente todas las reinas se emplea en generar valores pseudoaleatorios. Aun cuando nuestro rendimiento podría haber sido mejor si hubiéramos utilizado un generador más rápido —aunque menos sofisticado—¿debemos concluir que los beneficios de la aleatoriedad probablemente se compensan con coste adicional de la generación de pseudoaleatorios?

Una vez más, el problema de las ocho reinas se generaliza a un número de reinas n arbitrario. La ventaja ganada al utilizar la aproximación probabilística se hace más convincente a medida que crece n . Probamos el algoritmo en el problema de las 39 reinas. La vuelta atrás puramente determinista necesita examinar más de 10^{10} nodos antes de encontrar la primera solución: son 11.402.835.415 nodos para ser exactos. En comparación, el algoritmo de Las Vegas tiene éxito con una probabilidad aproximada del 21% si sitúa aleatoriamente las 29 primeras reinas, después de explorar solamente unos 100 nodos tanto si tiene éxito como si fracasa. Con el análisis en el caso medio, se exploran menos de 500 nodos antes de alcanzar el éxito si se reinicia el algoritmo después de cada fallo. Esto es más de 20 millones de veces mejor que la vuelta atrás. Además, la mejora persiste cuando se considera el tiempo en una computadora real: se necesitan aproximadamente 41 horas de cálculo ininterrumpido en nuestra estación de trabajo para resolver el problema mediante vuelta atrás puramente determinista, mientras que el algoritmo de Las Vegas encuentra una solución cada 8,5 milisegundos. Si se emplea el algoritmo voraz de Las Vegas puro, la tasa de aciertos es aproximadamente de uno entre cada 135 intentos, pero cada intento es tan rápido que se encuentra una solución aproximadamente cada 150 milisegundos de análisis en el caso medio, lo cual sigue siendo casi un millón de veces más rápido que la vuelta atrás pura.

Si deseamos una solución del problema de las n reinas para un valor concreto de n , es evidentemente tonto analizar exhaustivamente todas las posibilidades para descubrir el valor óptimo de *detenerLV*, y aplicar entonces el algoritmo de Las Vegas en consecuencia. La evidencia empírica sugiere que el algoritmo de Las Vegas supera ampliamente la vuelta atrás pura siempre y cuando situemos casi todas —pero no todas— las reinas aleatoriamente. Por ejemplo, se minimiza el número esperado de nodos si colocamos aleatoriamente 88 reinas para el problema de las 100 reinas, y 983 para el problema de las 1.000 reinas. Aun cuando no se utilice el mejor valor posible para *detenerLV*, obtenemos nuestra

solución con razonable rapidez si no está demasiado alejado. Además, es fácil obtener muchas soluciones diferentes sin más que invocar repetidamente el algoritmo.

10.7.2 Selección y ordenación probabilistas

Volvemos al problema de hallar el k -ésimo menor elemento de un vector T de n elementos. Vimos en la Sección 7.5 un algoritmo que puede resolver este problema en un tiempo que está en $\Theta(n)$ en el caso peor, independientemente del valor de k . En particular, al seleccionar $k = \lfloor n/2 \rfloor$ se obtiene un algoritmo de tiempo lineal en el caso peor para calcular la mediana de un vector.

Recuerde que este algoritmo empieza por particionar los elementos del vector a ambos lados de un pivote, y que después, tal como en la búsqueda binaria, restringe su atención al subvector correspondiente. El proceso se repite hasta que son iguales todos los elementos que aún quedan por considerar, quizá porque sólo quede uno, en cuyo caso habremos hallado el valor deseado. Un principio fundamental de la técnica de divide y vencerás sugiere que cuanto más próximo a la mediana de los elementos esté el pivote, más eficiente será el algoritmo. A pesar de esto, no hay ni que pensar en seleccionar la mediana exacta como pivote: esto daría lugar a una recursividad infinita, porque hallar la mediana es un caso especial del problema de selección que estamos considerando. Por tanto, seleccionamos un pivote subóptimo conocido con el nombre de pseudomediana. Esto evita la recursividad infinita, pero sigue siendo relativamente costoso hallar la pseudomediana.

Por otra parte, también vimos una aproximación más sencilla que utiliza como pivote el primer elemento que quede por considerar. Esto nos asegura un tiempo de ejecución lineal de análisis en el caso promedio, pero con el riesgo de que el algoritmo requiera un tiempo cuadrático en el caso peor. A pesar de este caso peor prohibitivo, el algoritmo más sencillo tiene la ventaja de una constante oculta mucho más reducida, como consecuencia del tiempo que se ahorra al no calcular la pseudomediana. Cualquier estrategia determinista sencilla para seleccionar el pivote tiene bastantes probabilidades de dar lugar a un tiempo cuadrático en el caso peor para hallar la mediana, y a la inversa, los algoritmos lineales en el caso peor parecen necesitar una constante oculta muy grande. La decisión concerniente a si es más importante tener una ejecución eficiente en el caso peor o análisis en el caso medio debe de tomarse a la vista de la aplicación concreta. Si decidimos tener una mayor velocidad de análisis en el caso promedio mediante un algoritmo determinista más sencillo, debemos asegurarnos de que los casos que haya que resolver sean realmente seleccionados según la distribución uniforme. Una mala distribución de probabilidades de los casos podría significar el desastre.

Para que el tiempo de ejecución dependa solamente del número de elementos, pero no del ejemplar en sí, basta con seleccionar aleatoriamente el pivote entre los elementos que queden por considerar. El algoritmo resultante es muy similar a selección de la Sección 7.5:


```

función selecciónLV( $T[1..n], s$ )
{Halla el  $s$ -ésimo elemento menor de  $T$ ,  $1 \leq s \leq n$ }
 $i \leftarrow 1; j \leftarrow n$ 
repetir
    {La respuesta se encuentra en  $T[i..j]$ }
     $p \leftarrow T[\text{uniforme}(i..j)]$ 
     $\text{pivotebis}(T[i..j], p, k, l)$ 
    si  $s \leq k$  entonces  $j \leftarrow k$ 
    sino si  $s \geq l$  entonces  $i \leftarrow l$ 
    sino devolver  $p$ 

```

El análisis solicitado en el problema 7.18 se aplica *mutatis mutandis* para concluir que el tiempo esperado que requiere este algoritmo probabilista de selección es lineal, independientemente del caso que haya que resolver. Por tanto, su eficiencia no se ve afectada por las peculiaridades de la aplicación en la cual se utilice el algoritmo. Siempre es posible que alguna ejecución particular del algoritmo requiera un tiempo cuadrático, pero la probabilidad de que suceda esto se vuelve cada vez más despreciable a medida que crece n y este suceso tan poco probable ya no está asociado a casos concretos.

Para resumir, hemos empezado con un algoritmo que es excelente cuando consideramos su tiempo medio de ejecución para todos los casos de algún tamaño dado, pero que es ineficiente en ciertos casos concretos. Empleando la aproximación probabilista, hemos transformado este algoritmo en un algoritmo de Las Vegas que es eficiente con una elevada probabilidad, sea cual fuere el caso considerado. De esta manera nos beneficiamos de los dos algoritmos deterministas vistos en la Sección 7.5: tenemos un tiempo esperado lineal en todos los casos, con una constante oculta reducida.

En cierta ocasión, indicamos a los alumnos de un curso de algoritmia que implementasen el algoritmo de selección que mejor les pareciera. Los únicos algoritmos vistos en clase eran los de la Sección 7.5. Dado que los alumnos no sabían qué ejemplares se iban a utilizar para probar sus programas —y temiendo lo peor por parte de sus profesores— ninguno se atrevía a correr el riesgo de emplear un algoritmo determinista con un caso peor cuadrático. Sin embargo, tres alumnos pensaron en emplear la aproximación probabilista. Esta idea les permitió derrotar a sus compañeros sin ningún esfuerzo: sus programas requerían una media de 300 milisegundos para resolver el caso trivial, mientras que la mayoría de los algoritmos deterministas requerían entre 1.500 y 2.600 milisegundos. Además, sus programas eran mucho más sencillos —y por tanto, menos proclives a contener errores sutiles— que los de sus compañeros.

Este mismo enfoque se puede emplear para transformar *quicksort* en un algoritmo que ordene n elementos en un tiempo esperado para el caso peor que está en $O(n \log n)$, mientras que el algoritmo que vimos en la Sección 7.4.2 requiere un tiempo que está en $\Omega(n^2)$ cuando el vector que hay que ordenar ya está ordenado. La versión aleatorizada de *quicksort* es como sigue. Para ordenar todo un vector T , basta llamar a *quicksortLV*($T[1..n]$):

```

procedimiento quicksortLV( $T[i, j]$ )
  {Ordena el subvector  $T[i, j]$  en orden no decreciente}
  si  $j-i$  es suficientemente pequeño entonces insertar( $T[i, j]$ )
  sino
     $p \leftarrow T[\text{uniforme}(i..j)]$ 
    pivotebis( $T[i..j], p, k, l$ )
    quicksortLV( $T[i..k]$ )
    quicksortLV( $T[l..j]$ )

```

10.7.3 Tablas de dispersión universales

Recuerde de la Sección 5.6 que las tablas asociativas, como las que se utilizan para llevar la cuenta de los identificadores en un compilador, suelen implementarse por dispersión. Esto nos da un tiempo esperado constante de acceso a la tabla, siempre y cuando los símbolos de la tabla sean aleatorios. Desafortunadamente, esto no dice nada acerca del rendimiento de la dispersión en casos no aleatorios. Si por ejemplo, se emplea la dispersión para implementar la tabla de símbolos en un compilador, entonces la suposición de que todos los identificadores son igualmente probables no es razonable (¡afortunadamente!). Consiguientemente, ese análisis de caso medio puede inducir a confusión, y la probabilidad de los casos malos puede ser significativamente más alta que la esperada.

Hay algo más importante, y es que ciertos programas van a causar inevitablemente más colisiones que las esperadas sin que tenga la culpa el desafortunado programador. Estos programas se compilarán lentamente en todas las ocasiones, porque la función de dispersión se fija para siempre en el compilador. La sabiduría popular afirma que las cosas se igualan desde el punto de vista del sistema: algunos programas necesitarán más tiempo para compilarse que el esperado, pero otros irán más deprisa. Al final del día, la compilación habrá sido eficiente en el análisis del caso medio. Este punto de vista es inherentemente injusto. Si cada programa se compila muchas veces, siempre habrá unos pocos programas (y siempre serán los mismos) que requerirán un tiempo de compilación substancialmente mayor que el esperado. En un sentido real, estos programas pagan el coste para que todos los demás programas se compilen rápidamente. La dispersión de Las Vegas nos permite retener la eficiencia de la dispersión análisis en el caso medio, sin favorecer arbitrariamente a algunos programas a expensas de otros. Éste es el efecto Robin Hood en plena potencia: se le da a cada programa su justa proporción de los beneficios obtenidos por la dispersión. También, todos los programas pagarán de vez en cuando el precio de la eficiencia general, requiriendo un tiempo mayor que el esperado. Además, el buen rendimiento esperado de la dispersión de Las Vegas se puede demostrar matemáticamente sin hacer suposiciones acerca de la distribución de probabilidad de las secuencias de acceso a la tabla.

La idea básica de la dispersión de Las Vegas es que el compilador seleccione aleatoriamente la función de dispersión al principio de cada compilación, y que lo haga de nuevo cuando resulte necesaria una nueva dispersión. Esto asegura que

las listas de colisiones tienen una probabilidad alta de estar razonablemente bien equilibradas, sea cual fuere el conjunto de identificadores del programa que hay que compilar. Como resultado, un programa que dé lugar a un gran número de colisiones durante una compilación probablemente tendrá más suerte en la próxima ocasión en que se compile. Ahora bien, ¿qué quiere decir «seleccionar aleatoriamente la función de dispersión»?

La respuesta se encuentra en una técnica conocida con el nombre de *dispersión universal*. Sea U el universo de índices potenciales para la tabla asociativa, como puede ser el conjunto de todos los posibles identificadores si vamos a implementar un compilador, y sea $B = \{0, 1, 2, \dots, N-1\}$ el conjunto de índices de la tabla de dispersión. Consideremos en U dos x e y diferentes. Supongamos primero que $h: U \rightarrow B$ es una función seleccionada aleatoriamente de entre todas las funciones que van de U a B según la distribución uniforme. Entonces la probabilidad $h(x) = h(y)$ es $1/N$. Esto se debe a que $h(y)$ podría tomar cualquiera de los N valores de B con igual probabilidad; en particular, el valor atribuido a $h(x)$ también se tomaría para $h(y)$ con probabilidad $1/N$. Sin embargo, U suele ser grande y hay demasiadas funciones de U en B para que sea razonable seleccionar aleatoriamente una de ellas según la distribución uniforme.

Considérese ahora un conjunto H de funciones de U en B , y considérense de nuevo en U dos x e y diferentes. Supongamos que $h: U \rightarrow B$ es una función seleccionada aleatoriamente de entre los miembros de H según la distribución uniforme. Decimos que H es una *clase universal₂ de funciones de dispersión* si la probabilidad de que sea $h(x) = h(y)$ es como máximo $1/N$. En otras palabras, requerimos que la probabilidad de que $h(x) = h(y)$ sea pequeña independientemente de los valores diferentes de x e y que se hayan considerado, siempre y cuando la selección de h se haga independientemente de estos valores. Vimos que el conjunto de todas las funciones de U en B es *universal₂*, pero demasiado grande para resultar útil. Las clases universales son interesantes porque pueden ser razonablemente pequeñas, de tal manera que en la práctica se puede seleccionar aleatoriamente una función de una de estas clases según la distribución uniforme. Además, las funciones se pueden evaluar eficientemente. Daremos enseguida un ejemplo explícito de una de estas clases *universal₂* de funciones de dispersión, pero primero vamos a ver lo bien que funcionan para resolver el problema de la compilación.

Sea H una clase *universal₂* de funciones de dispersión de U en B . Sean x e y dos identificadores distintos cualesquiera. Por definición de universalidad, si seleccionamos aleatoriamente h dentro de H según la distribución uniforme, entonces la probabilidad de colisión entre x e y es como mucho de $1/N$. Considérese ahora un programa con m identificadores diferentes, y sea x uno cualquiera de ellos. Para cada uno de los $m-1$ identificadores distintos de x , la probabilidad de colisión con x es como mucho $1/N$. Por tanto, el número esperado de identificadores en colisión con x es como mucho $(m-1)/N$, que es menor que el factor de carga. Dado que esto es cierto para todo x , y puesto que mantenemos el factor de carga por debajo de uno por reas dispersión cuando es necesario, todo acceso a la tabla de dispersión requiere un tiempo esperado constante en el caso peor. Por tanto n accesos a la tabla de dispersión requieren un tiempo esperado en $\Theta(n)$ en el caso peor.

Compárese esto con el tiempo en $\Omega(n^2)$ que se requiere en el caso peor para la dispersión clásica. Tal como sucede con los algoritmos de Las Vegas, el «caso peor» hace alusión al peor conjunto posible de accesos a la tabla, y no a las peores selecciones aleatorias que pudiera hacer el compilador.

Se conocen varias clases $universal_2$ de funciones de dispersión. Daremos un ejemplo. Supongamos por sencillez que $U = \{0, 1, 2, \dots, a - 1\}$ y que $B = \{0, 1, 2, \dots, N - 1\}$. (Los identificadores deben transformarse en enteros empleando cualquier representación entera estándar de las cadenas de caracteres, como ASCII.) Sea p un número primo tan grande al menos como a . Sean i y j dos enteros. Definimos $h_{ij}: U \rightarrow B$ como

$$h_{ij}(x) = ((ix + j) \bmod p) \bmod N.$$

Entonces, $H = \{h_{ij} \mid 1 \leq i < p \text{ y } 0 \leq j < p\}$ es una clase $universal_2$ de funciones de dispersión de U en B . Seleccionar aleatoriamente una función en H es tan sencillo como seleccionar dos enteros menores que p . Además, se puede calcular eficientemente el valor de $h_{ij}(x)$, especialmente si hacemos que N sea una potencia de 2, lo cual simplifica la segunda operación del módulo.

10.7.4 Factorización de enteros muy grandes

Sea n un entero mayor que 1. El problema de *factorización* consiste en encontrar la descomposición única de n como producto de factores primos. El problema de *descomposición* consiste en hallar un divisor no trivial de n , siempre y cuando n sea compuesto. La factorización se reduce a la descomposición y a la comprobación de primalidad: para factorizar n , hemos acabado si n es primo; en caso contrario, se halla un divisor no trivial de m y se factorizan recursivamente m y n/m .

El algoritmo sencillo de descomposición es la prueba división, que halla el menor primo que sea divisor de n . Es inútil buscar divisores mayores que $\sqrt{n}$ porque si $m > \sqrt{n}$ divide a n , entonces también lo divide n/m , que es menor que $\sqrt{n}$:

```
función pruebadiv( $n$ )
  para  $m \leftarrow 2$  hasta  $\lfloor \sqrt{n} \rfloor$  hacer
    si  $m$  divide a  $n$ , entonces devolver  $m$ 
  {Si el bucle no encuentra un divisor,  $n$  es primo}
  devolver  $n$  {un número primo es su menor divisor primo}
```

Este algoritmo requiere un tiempo que está en $\Omega(\sqrt{n})$ en el caso peor, que no tiene ninguna aplicación práctica ni siquiera para enteros de tamaño medio: si contamos tan sólo un nanosegundo para cada pasada por el bucle, se necesitarían miles de años para descomponer un número compuesto duro con unos cuarenta dígitos decimales, donde «duro» quiere decir que el número es el producto de dos primos de tamaños aproximadamente iguales.

El mayor número compuesto duro que se ha factorizado en el momento de escribir esto tiene 129 dígitos decimales. Esta factorización fue la clave para enfrentarse al desafío criptográfico RSA mencionado en la Sección 7.10. Recuerde

que se necesitaron ocho meses de cálculos en más de 600 computadoras de todo el mundo. Se ha estimado que habrían sido necesarios 5.000 años de cálculo ininterrumpido si se hubiese empleado una única estación de trabajo capaz de efectuar un millón de instrucciones por segundo. Aunque el esfuerzo es enorme, el éxito no habría sido posible sin un algoritmo sofisticado. De hecho, cuando se hizo el desafío en 1977, se estimó que la computadora más rápida disponible en aquel momento, ejecutando el mejor algoritmo conocido entonces, habría finalizado el cálculo ¡al cabo de dos millones de veces la edad del Universo! En esta sección damos tan sólo un esbozo de la idea básica que subyace al algoritmo victorioso.

Los algoritmos eficientes de descomposición se basan en el teorema siguiente, cuya sencilla demostración dejamos como ejercicio.

Teorema 10.7.1. Sea n un número entero compuesto. Sean a y b dos enteros distintos entre 1 y $n - 1$ tales que $a + b \neq n$. Si $a^2 \bmod n = b^2 \bmod n$, entonces $\text{mcd}(a+b, n)$ es un divisor no trivial de n

Considérese por ejemplo $n = 2.537$. Sean $a = 2.012$ y $b = 1.127$. Obsérvese que $a^2 = 1595n + 1629$ y $b^2 = 500n + 1629$, tanto a^2 como b^2 son iguales a 1.629 módulo n . Dado que $a \neq b$ y que $a + b \neq n$, el teorema dice que $\text{mcd}(a+b, n) = \text{mcd}(1.319, 2.537) = 43$ es un divisor no trivial de n , lo cual es cierto. Esto sugiere un enfoque para descomponer n : buscar dos números distintos entre 1 y $n - 1$ que tengan el mismo cuadrado módulo n , pero cuya suma no sea n , y utilizar el algoritmo de Euclides para calcular el máximo común divisor de su suma con n . Esto está muy bien siempre y cuando existan tales números cuando n es compuesto, y siempre y cuando podamos hallarlos eficientemente.

La primera cuestión se resuelve rápidamente. Supuesto que n tenga al menos dos divisores primos diferentes, $a^2 \bmod n$ admite al menos cuatro «raíces cuadradas» distintas en aritmética módulo n para cualquier a relativamente primo con respecto a n . Prosiguiendo con nuestro ejemplo, 1.629 admite exactamente cuatro raíces cuadradas módulo 2.537, a saber, 525, 1.127, 1.410 y 2.012. Estas raíces vienen en parejas: $525 + 2.012 = 1.127 + 1.410 = 2.537$. Dos cualquiera de estas raíces servirán, siempre y cuando no vengan de la misma pareja.

¿Y entonces, cómo podemos hallar a y b con la propiedad deseada? Éste es el momento en que entra en juego lo aleatorio. Sea k un entero que especificaremos después. Un entero se llama k -uniforme³ si todos sus divisores primos se encuentran entre los k números primos más pequeños. Por ejemplo, $120 = 2^3 \times 3 \times 5$ es 3-suave, pero $35 = 5 \times 7$ no lo es. Cuando k es pequeño, los enteros k -suaves se pueden factorizar eficientemente mediante la prueba de la división. En su primera fase, el algoritmo de descomposición de Las Vegas selecciona aleatoriamente un entero x entre 1 y $n - 1$, y calcula $y = x^2 \bmod n$. Si y es k -suave, entonces tanto x como la factorización de y se almacenan en una tabla. En caso contrario, seleccionamos aleatoriamente otro x . El proceso se repite hasta que se hayan encontrado $k+1$ en-

teros diferentes para los cuales se conozca la factorización de sus cuadrados módulo n .

Prosiguiendo con nuestro ejemplo en que $n = 2.537$, tomemos $k = 7$. Entonces nos conciernen solamente los primos 2, 3, 5, 7, 11, 13 y 17. Seleccionamos aleatoriamente un primer entero $x = 1.769$. Calculamos su cuadrado módulo n : $x^2 = 1.233n + 1.240$ y por tanto $y = 1.240$. No tiene éxito el intento de factorizar $1.240 = 2^3 \times 5 \times 31$, puesto que 31 no es divisible por ninguno de los primos admisibles. Un segundo intento con $x = 2.455$ es más afortunado: su cuadrado módulo n es $1.650 = 2 \times 3 \times 5^2 \times 11$. En este ejemplo pequeño, los intentos tienen éxito con una probabilidad aproximada del 20%. Prosiguiendo hasta conseguir 8 éxitos, obtenemos la tabla siguiente

$$\begin{array}{ll} x_1 = 2.455 & y_1 = 1.650 = 2 \times 3 \times 5^2 \times 11 \\ x_2 = 970 & y_2 = 2.210 = 2 \times 5 \times 13 \times 17 \\ x_3 = 1.105 & y_3 = 728 = 2^3 \times 7 \times 13 \\ x_4 = 1.458 & y_4 = 2.295 = 3^3 \times 5 \times 17 \\ x_5 = 216 & y_5 = 990 = 2 \times 3^2 \times 5 \times 11 \\ x_6 = 80 & y_6 = 1.326 = 2 \times 3 \times 13 \times 17 \\ x_7 = 1.844 & y_7 = 756 = 2^2 \times 3^3 \times 7 \\ x_8 = 433 & y_8 = 2.288 = 2^4 \times 11 \times 13 \end{array}$$

Esta tabla se utiliza para formar una matriz M de dimensiones $(k+1) \times k$ sobre $\{0, 1\}$. Cada fila corresponde a un éxito; cada columna corresponde a uno de los primos admisibles. La entrada M_{ij} recibe como valor 0 si el j -ésimo primo aparece como potencia par (incluyendo el cero) en la factorización de y_i ; en caso contrario $M_{ij} = 1$. Por ejemplo, $M_{11} = 1$ porque el primer primo, 2, aparece con elevado a 3 en y_1 , y $M_{12} = 0$ porque el segundo primo, 3, aparece con la potencia par 0. Para seguir con nuestro ejemplo, obtenemos la matriz siguiente:

$$M = \begin{pmatrix} 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix}$$

Dado que esta matriz contiene más filas que columnas, las filas no pueden ser linealmente independientes: debe existir un conjunto no vacío de filas cuya suma valga el vector cero en aritmética módulo 2. Este conjunto se puede hallar mediante la eliminación de Gauss-Jordan, aunque están disponibles métodos más eficientes cuando k es grande: sobre todo para matrices muy dispersas tales como las

que se obtienen mediante este algoritmo de factorización cuando n es grande. En nuestro ejemplo, hay siete soluciones distintas, tales como las filas 1, 2, 4 y 8, o las filas 1, 3, 4, 5, 6 y 7. Considere ahora lo que sucede si se multiplican las y_i correspondientes a las filas seleccionadas. Nuestros dos ejemplos producen

$$\begin{aligned} y_1 y_2 y_4 y_8 &= 2^6 \times 3^4 \times 5^4 \times 11^2 \times 13^2 \times 17^2 \\ y_1 y_3 y_4 y_5 y_6 y_7 &= 2^8 \times 3^{10} \times 5^4 \times 7^3 \times 11^2 \times 13^2 \times 17^2 \end{aligned}$$

respectivamente. Los exponentes de estos productos son necesariamente pares por construcción. Por tanto, se puede obtener una raíz cuadrada de estos productos dividiendo por dos los exponentes.

En aritmética módulo n , se puede obtener una raíz cuadrada del mismo producto multiplicando los x_i correspondientes, puesto que cada $y_i = x_i^2 \bmod n$. En nuestro ejemplo, los dos enfoques para calcular una raíz cuadrada módulo n de $y_1 y_2 y_4 y_8$ dan lugar a

$$\begin{aligned} a &= 2^3 \times 3^2 \times 5^2 \times 11 \times 13 \times 17 \bmod 2.537 = 2.012 \\ b &= 2.455 \times 970 \times 1.458 \times 433 \bmod 2.537 = 1.127. \end{aligned}$$

Tal como vimos anteriormente, basta con calcular el máximo común divisor de $a + b$ y n para obtener un divisor no trivial de n . En general, esta técnica produce dos enteros a y b entre 1 y $n - 1$ tales que $a^2 \bmod n = b^2 \bmod n$. Sin embargo no hay garantía, de que $a \neq b$ ni de que $a + b \neq n$. De hecho, la utilización de $y_1 y_3 y_5 y_6 y_7$ en lugar de $y_1 y_2 y_4 y_8$ se obtiene

$$\begin{aligned} a' &= 2^4 \times 3^5 \times 5^2 \times 7 \times 11 \times 13 \times 17 \quad \bmod 2.537 = 1.973 \\ b' &= 2.455 \times 1.105 \times 1.458 \times 216 \times 80 \times 1.844 \bmod 2.537 = 564 \end{aligned}$$

que resulta inútil porque $a' + b' = n$. Sin embargo, se puede demostrar que todo este proceso tiene éxito con una probabilidad mínima del 50% a no ser que $\text{mcd}(a, n)$ sea un divisor no trivial de n , lo cual nos viene igualmente bien a efectos de la descomposición; véase el problema 10.41. Sin embargo al contrario del problema de las n reinas, no debemos volver a empezar partiendo de cero en caso de fallo. ¿Por qué vamos a descartar tanto trabajo bien hecho? En su lugar, buscaremos otros conjuntos de filas de M que sumen cero en aritmética módulo 2. Si esto tampoco funciona, buscaremos unos pocos pares más $\langle x_i, y_i \rangle$ y volveremos a probar con la matriz aumentada resultante.

Queda por determinar el valor de k que debe utilizarse para optimizar el rendimiento de esta aproximación. Cuanto mayor sea este parámetro, mayor será la probabilidad de que $x' \bmod n$ sea k -suave cuando seleccionemos x aleatoriamente. Por otra parte, cuanto más pequeño sea el parámetro, más deprisa podremos efectuar una prueba de k -suavidad, y más deprisa factorizaremos los valores k -suaves

que encontremos, y menos valores necesitaremos. La búsqueda de un compromiso óptimo requiere el uso de resultados avanzados de teoría de números. Sea

$$L = e^{\frac{1}{2} \log n \log \log n}$$

sea b un número real positivo arbitrario y sea $t = L^{1/2b}$. Se puede demostrar que si $k \approx L^b$, entonces aproximadamente un x de cada t es tal que $x^2 \bmod n$ es k -suave. Dado que cada intento que no tiene éxito requiere k divisiones, y que se necesitan $k+1$ éxitos para construir la matriz, esta fase requiere un número esperado de divisiones de prueba que está aproximadamente en $O(tk^2) = O(L^{2b+1/2b})$, que se minimiza en $O(L^2)$ cuando $b = \frac{1}{2}$. La búsqueda de un conjunto de filas que tengan como suma el vector cero requiere un tiempo que está en $O(k^3) = O(L^{3b})$ si se emplea la eliminación de Gauss-Jordan (una vez más, es posible mejorarlo), lo cual es despreciable en comparación con el tiempo que se necesita para construir la matriz si es $b = \frac{1}{2}$. El cálculo final de un máximo común divisor por el algoritmo de Euclides es totalmente despreciable. De esta manera, si tomamos $k \approx \sqrt{L}$, el algoritmo produce la descomposición de n al cabo de un número esperado de divisiones que está en $O(L^2)$. Si n es un número medio de 100 dígitos decimales, entonces $L \approx 5 \times 10^{30}$ mientras que $\sqrt{n} \approx 7 \times 10^{19}$, que es 10^{19} veces mayor. Esto ilustra lo mucho mejor que es este algoritmo en comparación con las pruebas de división. Esta comparación no es enteramente imparcial, puesto que la constante oculta para la prueba de la división es más pequeña, pero incluso 10^{30} picosegundos son aproximadamente el doble de la edad estimada del Universo.

Hay varias mejoras que hacen más práctico el algoritmo. Si en lugar de seleccionar x aleatoriamente entre 1 y $n-1$ lo seleccionamos de tal manera que $x^2 \bmod n$ tenga más probabilidades de ser k -uniforme, entonces reducimos el número de pruebas previas para obtener las $k+1$ relaciones necesarias. La aleatoriedad desempeña un papel fundamental en este algoritmo porque no hay ninguna aproximación determinista para hallar tantos x buenos que haya resultado ser eficiente. Sin embargo, hay heurísticas no demostradas que funcionan tan bien en la práctica que sería necio utilizar el algoritmo «puro» que se ha descrito más arriba. La más sencilla de éstas consiste en seleccionar los x ligeramente más grandes que $\sqrt{n}$; véase el Problema 10.42. Otra heurística, la *criba cuadrática*, funciona en un tiempo que está en $O(L)$. La factorización con éxito de un número *duro* de 129 dígitos decimales que se mencionaba al principio de esta sección se efectuó empleando un algoritmo algo más sofisticado: la «variante polinómica múltiple doble de la criba cuadrática para grandes primos». Otras técnicas de factorización interesantes son el método de la curva elíptica y la criba de cuerpos numéricos que exigen resultados relativamente avanzados de la teoría de números.